

基于联合物理色散模型与先进信号处理方法测定碳化硅外延层厚度

摘 要

本文针对红外干涉法测量半导体外延层厚度问题，建立了基于物理原理的双光束及多光束数学模型。我们为碳化硅和硅两种材料分别构建了针对性的折射率模型，并设计了相应的外延层厚度求解算法。通过对光谱数据进行分析，我们成功求解出其外延层厚度，并基于结果揭示了样品的物理性质及多光束干涉效应的影响。

针对问题一，为精确确定碳化硅外延层的厚度，我们建立了两个核心模型。一：采用基于**菲涅尔方程的双光束干涉理论**来描述光波在“空气-外延层-衬底”三层结构中的干涉过程；二：针对碳化硅的色散特性，我们引入了能同时描述晶格振动与自由载流子影响的**洛伦兹-德鲁德 (Lorentz-Drude) 色散模型**。

针对问题二，我们采用问题一建立的数学模型，通过拟合实验光谱来反演样品七个未知的物理参数。为此，我们设计了“**差分进化全局搜索**”与“**L-BFGS-B 局部优化**”相结合的算法，实现了对两组光谱数据的同步拟合。模型取得了极佳的拟合优度，决定系数 R^2 分别高达 **0.9967 和 0.9948**，最终确定外延层厚度为 **7.413 μm** 。基于拟合出的参数，我们发现外延层与衬底折射率高度匹配，从而证明了在该体系中多光束干涉效应可以忽略。最后，我们根据残差分析确认了结果的可靠性。

针对问题三，我们首先推导了产生多光束干涉的必要条件：相干性、界面反射率与材料吸收、以及几何均匀性，并分析出多光束干涉将有效提高硅晶圆片外延层厚度计算精度。然后，我们再对附件 3 和附件 4 的硅晶圆片光谱数据进行分析，通过干涉条纹的非对称性，先定性判断发生多光束干涉。针对硅的特性，我们采用**塞尔迈耶方程**描述折射率，并通过**干涉极值点解析法**，识别光谱极值点位置并反解出厚度，计算出硅晶圆片厚度为 **3.751 μm** 。为了定量证明发生了多光束干涉，我们通过反射率、介质折射率反推出衬底折射率，利用菲涅尔公式，定量计算出了各级反射光振幅、光谱精细度 ($F \approx 2.7 > 2$) 等数值，严谨证明了**多光束干涉效应的存在**。然而，我们发现干涉极值点法存在局限性：在高波数区，由于信号衰减，极值点定位精度下降。为克服此问题并利用全谱信息，我们引入了**基于色散校正的傅里叶变换 (FFT) 频率分析法**，并创新性地引入**坐标变换**来解决硅的色散现象导致的干涉信号的非周期性，将信号转化为一个在新坐标域中周期性厚度严格对应的信号，利用 FFT 从光谱数据中提取出基频，得到更准确的硅晶圆片厚度结果 **d = 3.595 μm** 。

此外，为了判定多光束干涉对问题二碳化硅外延层计算精度的影响，我们运用问题三推导的多光束干涉判据进行分析，最终确认**碳化硅无需考虑多光束干涉**，因此问题二采用的双光束干涉模型是充分且精确的。

综上所述，本文所建模型经过物理严格推导且针对性强，使用的算法精度高，准确计算了碳化硅和硅外延层的厚度测定，还通过灵敏度与稳定性检验证明了模型的稳健性，可为相关领域的精确测量提供科学依据。

关键词：外延层厚度；红外干涉光谱；洛伦兹-德鲁德模型；差分进化；L-BFGS-B；塞尔迈耶模型；傅里叶变换；色散校正

1 问题背景

碳化硅作为一种新兴的第三代半导体材料，以其优越的综合性能表现正在受到越来越多的关注。碳化硅外延层的厚度是外延材料的关键参数之一，对器件性能有重要影响。因此，制定一套科学、准确、可靠的碳化硅外延层厚度测试标准显得尤为重要。红外干涉法是外延层厚度测量的无损测量方法，可根据红外光谱的波长、外延层的折射率和红外光的入射角等参数确定外延层的厚度。针对红外干涉法测量外延层长度建立模型后，可以确定反射光谱与外延层厚度等参数之间的关系，可以在实际生产中用于评估外延层的质量和性能是否符合应用要求、最终产品的可靠性、性能和良率。

2 问题重述

- 问题一：红外光入射到外延层后，一部分从外延层表面反射出来，另一部分从衬底表面反射回来，这两束光在一定条件下会产生干涉条纹。问题一限制了条件，只需考虑外延层和衬底界面只有一次反射、透射所产生的干涉条纹的情形，要求建立确定外延层厚度的数学模型。问题还明确指出外延层的折射率不是常数，它与掺杂载流子的浓度、红外光谱的波长等参数有关。
- 问题二：该问题要求根据问题一建立的数学模型，设计确定外延层厚度的算法，基于附件给定的实测数据，计算碳化硅晶圆片外延层的厚度。
- 问题三：问题增加了光波可以在外延层界面和衬底界面产生多次反射和透射的条件，需要推导产生多光束干涉的必要条件，并且据此判断硅晶圆片是否产生了多光束干涉，以此为基础重新建立计算外延层厚度的数学模型，设计确定外延层厚度的算法，并基于附件给定的实测数据，计算硅晶圆片外延层的厚度，最后分析多光束干涉对硅晶圆片外延层厚度计算精度可能产生的影响。同时，该问题要求考虑问题一、二中多光束干涉对测定结果的影响，设法消除可能的影响，并给出消除影响后的计算结果。

3 问题分析

3.1 问题一的分析

问题一属于数学模型建立，为确定外延层厚度，我们需要建立两个物理模型来精确模拟红外光的反射光谱。一：因为探测到的反射光是两束光干涉叠加的结果，干涉的效果取决于两束光之间的相位差，与外延层的厚度直接相关，所以通过分析光谱中的干涉条纹我们就能反推出厚度。为了精确计算每次反射和透射的光强，我们需要引入菲涅尔方程分别处理 s 偏振光和 p 偏振光。二：构建材料色散模型。材料的折射率由于色散会随光的波长变化。为了描述这种变化，我们需要采用洛伦兹-德鲁德模型来全面地描述红外光与材料相互作用的两种主要物理机制，即晶格振动和自由载流子。然后我们就可以得到复数折射率关于波数的函数表达，其实部是传统意义上的折射率。然后我们将第二步中得到的复数折射率表达式代入第一步的光学干涉模型中，以此高精度地模拟出在整个红外波段内的完整反射光谱。

3.2 问题二的分析

问题二需要基于问题一建立的模型，通过设计拟合算法计算外延层厚度。我们采用双光束干涉模型与洛伦兹-德鲁德色散模型相结合的策略，寻找一组唯一的物理参数（包括外延层厚度、纵向声子阻尼等），使其能够同时模拟 10° 和 15° 两个不同角度的光谱数据。我们需要设计全局搜索和局

部优化算法，模拟目标参数的各种组合，通过最小化理论光谱与实验光谱的残差和，找到能够拟合实验光谱的最优参数组合来得到数据结果，并对比由这组参数生成的理论光谱和实验数据与物理现实以判断拟合效果，分析结果的可靠性。

3.3 问题三的分析

问题三是问题一二的延伸，针对问题三，我们首先推导了产生多光束干涉的必要条件。通过将多光束干涉理论模型的反射率曲线特征与附件 3 和 4 的硅晶圆光谱进行对比。我们在确定硅外延层厚度时利用了硅材料在红外波段高度透明且光学特性简单的特点，选择了两种比全局拟合更直接高效的算法。第一种是干涉极值点解析法，该方法通过输入光谱中的波峰波谷位置，并结合硅折射率色散的塞尔迈耶方程计算得出厚度，再通过遍历搜索算法确定了正确的干涉级数。为了进一步提高精度并利用全谱信息，我们还采用了第二种更为精密的傅里叶变换算法。该算法的核心在于通过一次精确的坐标变换，将因材料色散导致的非周期性干涉信号，转换为一个真正周期性的信号，从而消除了直接 FFT 会引入的系统误差。在此基础上，我们结合窗函数抑制和高精度峰定位技术，算法再次从两组数据中计算得到厚度。最后，我们需要通过对比干涉极值点法和傅里叶变换算法的结果，判断多光束干涉对外延层厚计算精度可能产生的影响。

4 模型假设

为了简化问题，便于模型的建立和求解，我们做出如下合理假设：

1. 假设本题中的碳化硅可以视作标准的 4H-SiC 材料，可以用洛伦兹-德鲁德色散模型精确描述。
2. 假设硅外延层的光学响应，可以被三项塞尔迈耶方程精确描述。
3. 假设用于计算的 4H-SiC 的本征光学参数（如 $\varepsilon_\infty, \omega_L, \omega_T$ ）以及硅的塞尔迈耶色散系数适用于本问题中的样品。
4. 假设入射的红外光为理想的非偏振平面波，并且其相干长度远大于光在薄膜中的最大光程差（ $L_c \gg 2nd$ ），可以产生稳定干涉条纹。
5. 假设入射的红外光为严格平行光。
6. 假设外界为空气或真空，折射率 $n_0 = 1$ 。
7. 假设探测器接收孔径和方向合适，能正确测得题目模型考虑的反射光强度。

5 符号说明

本文建模与计算过程中使用的主要符号及其含义如表 1 所示。

表 1 本文主要符号说明

符号	含义	单位
通用光学参数		
d	外延层厚度	μm
λ	真空中光的波长	μm
σ	波数 ($\sigma = 1/\lambda$)	cm^{-1}
$\theta_0, \theta_1, \theta_2$	在空气、外延层、衬底中的光线角度	度
n_0, n_1, n_2	空气、外延层、衬底的折射率实部	(无量纲)
$\tilde{n}$	复折射率 ($\tilde{n} = n + ik$)	(无量纲)
δ	干涉相位差	弧度
r_s, r_p	s 偏振和 p 偏振的振幅反射系数	(无量纲)
t_s, t_p	s 偏振和 p 偏振的振幅透射系数	(无量纲)
R	反射率	(无量纲)
洛伦兹-德鲁德模型参数 (SiC)		
$\varepsilon(\sigma)$	复介电函数	(无量纲)
ε_∞	高频介电常数	(无量纲)
ω_L, ω_T	纵向/横向光学声子频率	cm^{-1}
γ_L, γ_T	声子阻尼系数	cm^{-1}
ω_p	等离子体频率	cm^{-1}
γ_p	等离子体阻尼 (载流子散射率)	cm^{-1}
塞尔迈耶模型参数 (Si)		
B_j, C_j	塞尔迈耶方程色散系数	(无量纲), μm^2
傅里叶变换法参数		
$x(\sigma)$	用于线性化相位的坐标变换变量	cm^{-1}

6 问题一：数学模型的建立

为了确定碳化硅外延层的厚度，我们首先构建一个能够精确描述红外光与“空气-外延层-衬底”三层光学系统相互作用的数学模型。该模型基于电磁波的干涉理论和菲涅尔方程，并引入了先进的色散模型以描述材料光学常数随频率的变化。

6.1 干涉模型：双光束干涉的数学描述

根据题目图 1 所示的物理情景，我们考虑两束主要反射光线的干涉：光束 1（由空气/外延层界面直接反射）和光束 2（穿过外延层，由外延层/衬底界面反射后，再次穿出外延层）。

6.1.1 偏振光理论

光场的能量主要由电场携带，根据电磁波理论，电场方向垂直于光线传播方向。由于本题未提及光的偏振状态，在处理的时候默认为非偏振光，可视为包含大量的、不同取向、彼此不相关的线

偏光。其在任意两个垂直方向的振幅 A_0 与光强 I_0 有表达式:

$$I_0 = 2A_0^2 \quad (1)$$

经过透射与反射后,对平行于入射面的光(p光)和垂直于入射面的光(s光)非相干叠加,得到反射总光强表达式:

$$I_r = I_s + I_p \quad (2)$$

6.1.2 相位差与光强叠加

建立数学模型,仅考虑在空气-外延层界面直接反射光1和在外延层-衬底界面经过一次反射、透射的反射光2,二者在无穷远处发生干涉。

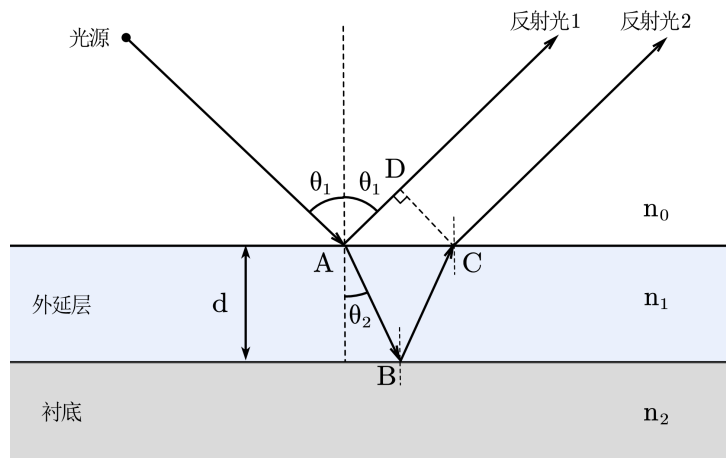


图 1 双光束干涉光路图

考虑反射光 1 和反射光 2 的光程差:

$$L = L_{AB} + L_{BC} - L_{AD} \quad (3)$$

由几何关系可以得到:

$$L_{AB} + L_{BC} = \frac{2n_1 d}{\cos \theta_2} \quad (4)$$

$$L_{AD} = 2n_0 d \tan \theta_2 \sin \theta_1 \quad (5)$$

带入总光程差表达式,得到:

$$L = 2n_1 d \cos \theta_2 \quad (6)$$

两束相干光叠加后的总光强取决于它们之间的相位差 δ 。该相位差由光程差和反射时可能发生的相位突变共同决定:

$$\delta = \frac{4\pi n_1 d \cos \theta_2}{\lambda_0} + \Phi_{\text{sub}} - \Phi_{\text{air}} \quad (7)$$

其中, d 是外延层的厚度, λ_0 是真空中光的波长, n_1 是外延层的折射率, θ_2 是光在外延层中的折射角。 Φ_{sub} 和 Φ_{air} 分别是光在外延层/衬底界面和空气/外延层界面反射时产生的相位突变。

假定入射光为非偏振光,其振幅可以分解为相互垂直的 s 偏振(垂直于入射面)和 p 偏振(平行于入射面)两个分量。设入射光的 s 偏振和 p 偏振分量振幅均为 A_0 ,则总入射光强为 $I_0 = 2A_0^2$ 。

两束反射光干涉后，s 偏振和 p 偏振的总反射光强 I_s 和 I_p 分别为：

$$I_s = A_{1s}^2 + A_{2s}^2 + 2A_{1s}A_{2s} \cos(\delta_s) \quad (8)$$

$$I_p = A_{1p}^2 + A_{2p}^2 + 2A_{1p}A_{2p} \cos(\delta_p) \quad (9)$$

其中， A_{1s}, A_{1p} 和 A_{2s}, A_{2p} 分别是光束 1 和光束 2 的 s 偏振和 p 偏振振幅。总反射光强为 $I_r = I_s + I_p$ ，最终的反射率 R 定义为：

$$R = \frac{I_r}{I_0} = \frac{I_s + I_p}{2A_0^2} \quad (10)$$

6.1.3 振幅计算：菲涅尔方程的应用

根据菲涅尔公式，可以得到反射与透射的 p 光和 s 光振幅与入射振幅关系。各反射光束和透射

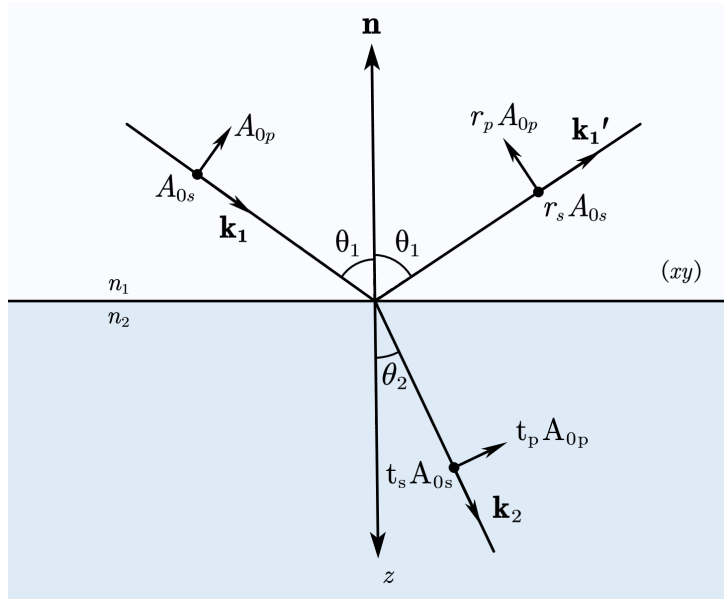


图 2 光在介质界面反射折射的基本物理图像

光束的振幅由描述电磁波在不同介质界面行为的菲涅尔方程确定。对于从介质 i 入射到介质 j 的情况，其振幅反射系数 (r) 和透射系数 (t) 为：

$$r_s = \frac{n_i \cos \theta_i - n_j \cos \theta_j}{n_i \cos \theta_i + n_j \cos \theta_j} \quad t_s = \frac{2n_i \cos \theta_i}{n_i \cos \theta_i + n_j \cos \theta_j} \quad (11)$$

$$r_p = \frac{n_j \cos \theta_i - n_i \cos \theta_j}{n_j \cos \theta_i + n_i \cos \theta_j} \quad t_p = \frac{2n_i \cos \theta_i}{n_j \cos \theta_i + n_i \cos \theta_j} \quad (12)$$

其中， n_i, n_j 和 θ_i, θ_j 分别是两介质的折射率和光线角度，它们通过斯涅尔定律 $n_i \sin \theta_i = n_j \sin \theta_j$ 相关联。

基于此，我们可以精确计算光束 1 和光束 2 的振幅。设空气折射率为 $n_0 = 1$ ，外延层折射率为 n_1 ，衬底折射率为 n_2 。

- 光束 1 (空气/外延层界面反射):

$$A_{1s} = r_s^{(0 \rightarrow 1)} A_0 = \frac{n_0 \cos \theta_0 - n_1 \cos \theta_1}{n_0 \cos \theta_0 + n_1 \cos \theta_1} A_0 \quad (13)$$

$$A_{1p} = r_p^{(0 \rightarrow 1)} A_0 = \frac{n_1 \cos \theta_0 - n_0 \cos \theta_1}{n_1 \cos \theta_0 + n_0 \cos \theta_1} A_0 \quad (14)$$

- **光束 2 (空气 → 外延层透射 → 衬底反射 → 外延层 → 空气透射)**: 其振幅是多次透射和一次反射过程的乘积:

$$A_{2s} = t_s^{(0 \rightarrow 1)} r_s^{(1 \rightarrow 2)} t_s^{(1 \rightarrow 0)} A_0 \quad \text{和} \quad A_{2p} = t_p^{(0 \rightarrow 1)} r_p^{(1 \rightarrow 2)} t_p^{(1 \rightarrow 0)} A_0 \quad (15)$$

将上述振幅表达式代入光强公式, 即可得到总反射率 R 作为外延层厚度 d 、波长 λ 、入射角 θ_0 以及各层折射率的函数。

6.2 色散模型: 材料光学常数的精确描述

题目指出, 外延层的折射率 n 并非一个常数。我们通过观察附件 1 和 2 的实验光谱, 发现实验光谱存在“剩余射线带”, 传统的塞尔迈耶方程无法描述此特质。为了精确地模拟光谱, 在 AI[1] 的搜索功能帮助下, 我们采了用因式分解形式的洛伦兹-德鲁德 (Lorentz-Drude) 色散模型。该模型通过复介电函数 $\varepsilon(\omega)$ 来描述材料的光学响应, 其表达式由文献 [2] 给出:

$$\varepsilon(\omega) = \varepsilon_\infty \left(\frac{\omega_L^2 - \omega^2 - i\gamma_L\omega}{\omega_T^2 - \omega^2 - i\gamma_T\omega} \right) - \frac{\omega_p^2}{\omega^2 + i\gamma_p\omega} \quad (16)$$

该模型结合了两种物理机制: 第一项 (洛伦兹振子项) 描述了由晶格振动 (声子) 引起的共振吸收; 第二项 (德鲁德项) 则描述了由掺杂引入的自由载流子的响应。

德鲁德项中的等离子体频率 ω_p 和阻尼 γ_p 与材料的关键电学性质——载流子浓度 N 和迁移率 μ 直接相关:

$$\omega_p^2 = \frac{Nq^2}{\varepsilon_0 m^*} \quad (17)$$

$$\gamma_p = \frac{q}{m^* \mu} \quad (18)$$

最终, 材料复折射率 $\tilde{n}(\omega) = n(\omega) + ik(\omega)$ 可通过以下关系式由复介电函数得到:

$$\tilde{n}(\omega) = \sqrt{\varepsilon(\omega)} \quad (19)$$

其中, 实部 $n(\omega)$ 为我们需要的折射率, 虚部 $k(\omega)$ 为消光系数。由于折射率是复数, 菲涅尔系数和相位差的计算都将在复数域中进行, 这自然地包含了吸收和相位突变效应。

7 问题二的模型建立与求解

问题二中, 我们使用了问题一的双光束干涉模型以及洛伦兹-德鲁德色散模型, 并根据文献确定了模型的部分基本参数, 以附件 1 和 2 的实验数据为基准, 对模型中未知的七个关键参数进行全局优化拟合, 从而解出一组最优参数, 最终确定外延层的厚度。

7.1 决策变量的选取

外延层和衬底的复介电函数 $\varepsilon(\sigma)$ 由公式 (16) 描述。

由题意, 外延层和衬底为同种材质, 但由于掺杂载流子浓度的不同而有不同的折射率。因此外延层和衬底的介电系数共享相同的晶格参数 ($\varepsilon_\infty, \omega_L, \omega_T, \gamma_L, \gamma_T$)。而载流子浓度的不同影响到材料的自由载流子响应项和迁移率, 反映在外延层和衬底有各自的德鲁德参数 (ω_{p1}, γ_{p1} 和 ω_{p2}, γ_{p2})。

因此, 求解本问的决策变量包含:

$$\{d, \gamma_L, \gamma_T, \omega_{p1}, \gamma_{p1}, \omega_{p2}, \gamma_{p2}\} \quad (20)$$

7.2 求解模型的数学框架

我们的任务是解决一个逆问题，即通过拟合实验光谱 $R_{\text{exp}}(\sigma)$ 来反向拟合出样品未知的物理参数。具体来说，我们设定了 7 个待拟合物理参数，需要通过固定物理参数计算出理论反射光谱，和实验光谱数据进行残差分析来计算出残差最小时对应的参数组合。

首先，我们假设样品为 4H-SiC 材料，参考文献中的物理参数，从而简化模型的计算。经过 AI[1] 搜索，该材料的本征光学参数可以依据文献 [2] 设定，具体如表 (2) 所示。

表 2 4H-SiC 材料固定的本征光学参数

参数	符号	数值	单位
高频介电常数	ε_{∞}	6.56	(无量纲)
纵向光学声子频率	ω_L	970.0	cm^{-1}
横向光学声子频率	ω_T	798.0	cm^{-1}

我们的算法模型目标是求解出最优的一组参数向量 $\mathbf{p}$ ，它们与介电函数紧密相关：

$$\mathbf{p} = \{d, \gamma_L, \gamma_T, \omega_{p1}, \gamma_{p1}, \omega_{p2}, \gamma_{p2}\} \quad (21)$$

这些参数通过以下物理链条决定最终的反射率：

1. **光学模型决定介电函数：**外延层和衬底的复介电函数 $\varepsilon(\sigma)$ 由公式 (16) 描述。其中，外延层和衬底共享相同的晶格参数 $(\varepsilon_{\infty}, \omega_L, \omega_T, \gamma_L, \gamma_T)$ ，但具有不同的德鲁德项参数 $(\omega_{p1}, \gamma_{p1}$ 和 $\omega_{p2}, \gamma_{p2})$ 。
2. **介电函数决定光学常数与界面行为：**复折射率 $\tilde{n}(\sigma) = \sqrt{\varepsilon(\sigma)}$ 决定了光在空气/外延层和外延层/衬底界面上的反射与透射行为，这些行为由菲涅尔方程精确描述。
3. **厚度决定干涉效应：**外延层的厚度 d 决定了两束反射光之间的相位差 δ ，关系见公式 (7)。最终的非偏振光反射率 R 由此确定。

我们需要寻找最优的 $\mathbf{p}_{\text{opt}}$ ，使得理论反射率 $R_{\text{theory}}(\sigma; \mathbf{p})$ 与实验数据 $R_{\text{exp}}(\sigma)$ 的差异最小。问题二求解的算法流程图如图 (3)。

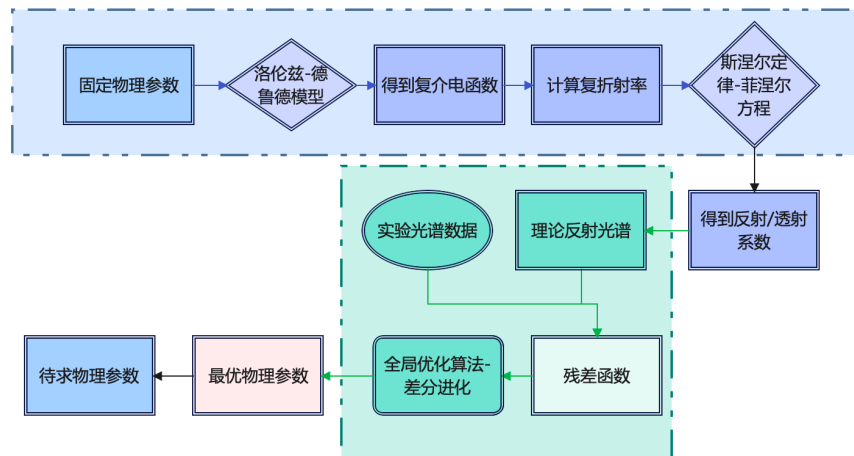


图 3 问题二求解算法流程图

7.3 算法设计

7.3.1 优化策略与目标函数

经过和 AI[3] 的讨论，我们决定采用全局同步拟合算法，即寻找一组唯一的物理参数向量 $\mathbf{p}$ ，使其能够同时最优地解释附件 1 和附件 2 两个独立的实验数据，所以我们设定目标函数为两个数据集均方误差 (MSE) 的总和：

$$\chi_{\text{total}}^2(\mathbf{p}) = \text{MSE}(\mathbf{p}, 10^\circ) + \text{MSE}(\mathbf{p}, 15^\circ) \quad (22)$$

此策略采用了一组唯一参数来同时约束两个不同角度的光谱数据，极大地降低了多参数逆问题中普遍存在的解不唯一风险，保证了核心厚度参数 d 的鲁棒性。

7.3.2 优化执行步骤

我们执行两步优化流程，以确保能够找到全局最优解，避免陷入局部极值。

1. **第一步：全局搜索。**我们使用差分进化算法 (Differential Evolution)，在预设的参数边界内进行并行搜索。
2. **第二步：局部优化。**在全局搜索找到一个接近最优解的区域后，我们以差分进化算法的输出作为初始值，采用 **L-BFGS-B 算法**进行精细的局部优化，搜索出更加精确的结果。

7.4 求解过程与结果分析

7.4.1 数据处理与算法参数设定

首先，我们注意到附件中的第一行数据显示反射率为 0，不符合物理事实，因此予以剔除，从第二个数据点开始进行拟合。其次，我们使用 AI[4] 生成了基本的优化算法，并且自行调整参数，对于第一步的**全局优化阶段（差分进化）**，我们设定种群大小为 40，最大迭代次数为 1000 次；同时固定随机种子为 42 以保证结果的可复现性。在随后的**局部优化阶段（L-BFGS-B）**，我们设定最大迭代次数为 500 次，并将收敛容差 (ftol, gtol) 设为严格的 1×10^{-12} 。

7.4.2 全光谱范围全局同步拟合

我们将附件 1 和附件 2 提供的完整光谱数据代入上述求解算法，进行全局同步拟合。最终得到的唯一参数组 p 如表 (3) 所示。

表 3 全光谱范围 (400 cm^{-1} to 4000 cm^{-1}) 全局同步拟合参数

参数	符号	拟合值
外延层厚度	d	$7.413 \mu\text{m}$
声子阻尼 (纵向)	γ_L	0.100 cm^{-1}
声子阻尼 (横向)	γ_T	1.096 cm^{-1}
外延层等离子体频率	ω_{p1}	457.42 cm^{-1}
外延层等离子体阻尼	γ_{p1}	1294.51 cm^{-1}
衬底等离子体频率	ω_{p2}	1120.07 cm^{-1}
衬底等离子体阻尼	γ_{p2}	648.05 cm^{-1}

相应的拟合曲线与实验数据对比如图（4）和（5）所示。从图中可以看出，附件 1 和附件 2 的决定系数 (R^2) 分别高达 **0.9967** 和 **0.9948**，证明了模型的高度准确性。

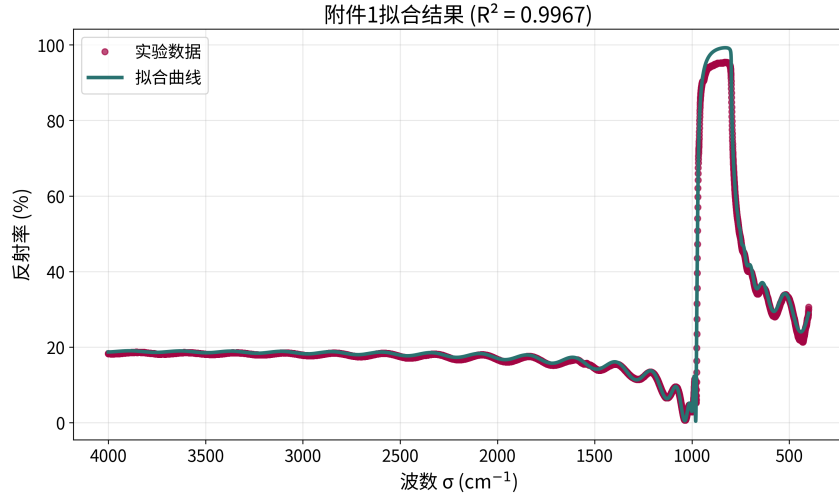


图 4 附件 1 全光谱范围的全局同步拟合结果 ($R^2=0.9967$)。

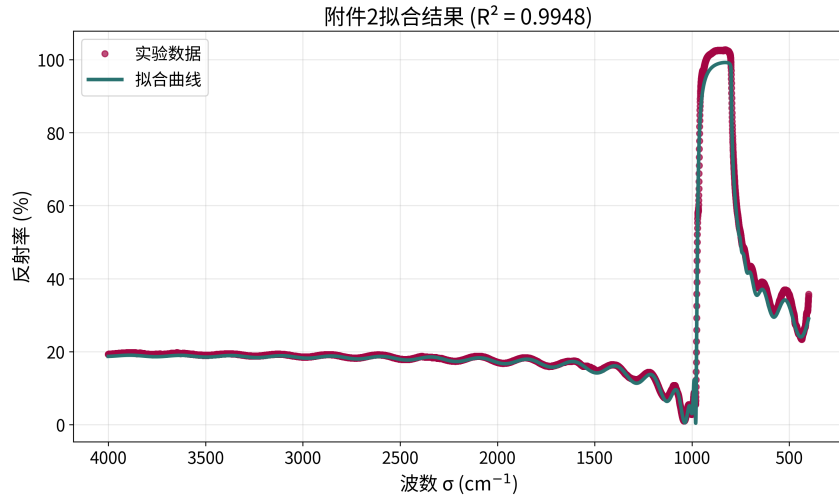


图 5 附件 2 全光谱范围的全局同步拟合结果 ($R^2=0.9948$)。

7.4.3 模型合理性分析与多光束干涉判断

为了验证我们所选双光束模型的合理性，我们基于表 3 中的参数组，计算了外延层和衬底的复折射率，如图（6）所示。

我们从图（6）中可以发现，在整个光谱范围内，外延层和衬底的折射率实部 (n) 几乎一致。根据菲涅尔方程，光在两种介质界面处的反射强度主要由二者折射率的差异决定。当 $n_1 \approx n_2$ 时，后续光束在经历多次弱界面反射后，其强度会迅速衰减至可以忽略不计，因此对于本问题中的碳化硅样品，我们可以忽略多光束干涉效应，可以采用**双光束干涉模型**这一物理模型。

7.4.4 最终结论与可靠性分析

基于以上分析，我们忽略了碳化硅样品的多光束干涉效应，采用更合理的双光束干涉模型。所以我们基于问题一模型计算出的厚度 $7.413\ \mu\text{m}$ 即为最终结果，无需进行额外修正。

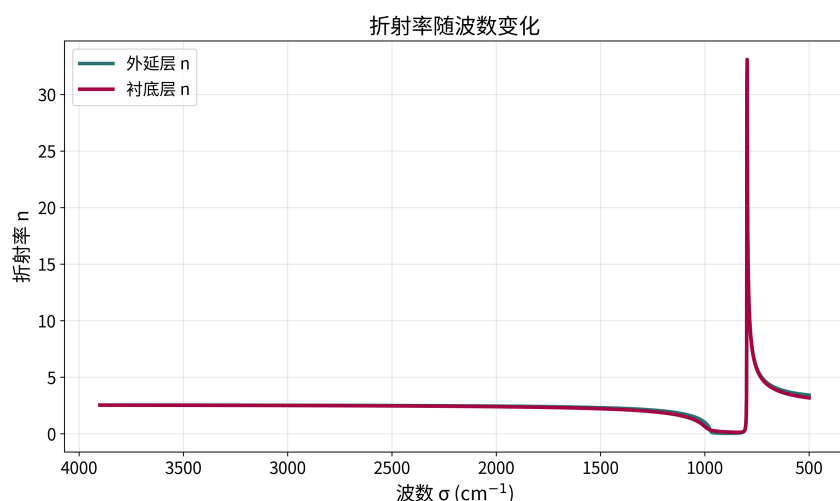


图 6 根据最终拟合参数计算得到的外延层与衬底的复折射率谱。

对于最终结果，我们分析其可靠性：

我们设计的算法成功在附件 1 和 2 的数据上获得了超过 0.99 的决定系数 (R^2)，说明我们的拟合结果有很强的精确度；我们绘制了拟合残差图，如图 (7) 和 (8)，在波数大于 1000 cm^{-1} 的区域，残差极小，表明模型在此区域的拟合效果很好。所以我们可以认为我们现有模型拟合出来的结果是可靠的。

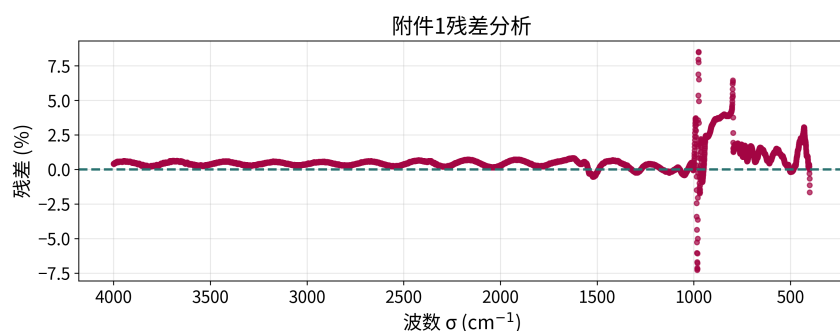


图 7 全光谱拟合中附件 1 的残差分布图。

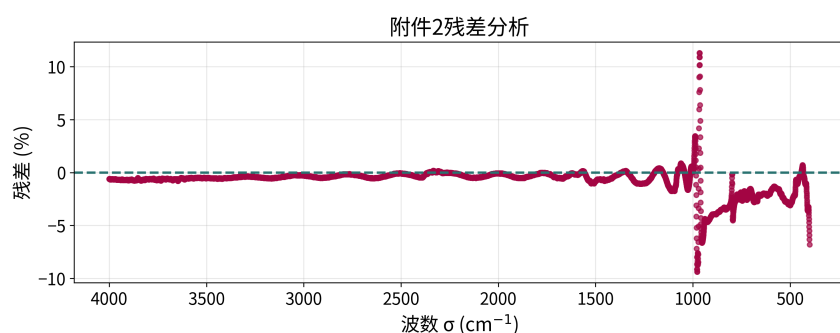


图 8 全光谱拟合中附件 2 的残差分布图。

当然，如上图所示，模型在低波数区域残差较大，这引发了我们对洛伦兹-德鲁德模型拟合碳化硅衬底的精确性的怀疑。查阅文献 [5] 后，我们认为残差图在低波数段的系统性偏差，并非简单的模型缺陷，而是高掺杂 SiC 衬底中 LO 声子-等离激元耦合 (LOPC) 现象的外显。

8 问题三模型的建立与求解

为了分析硅晶原片的测试结果是否出现多光束干涉，并计算出硅外延层的厚度，我们考虑一个由“空气-外延层-衬底”构成的 Fabry-Pérot 腔。该模型基于多光束干涉的经典理论和菲涅尔公式对反射率进行计算，并引入了针对硅晶体色散特性的折射率模型。

8.1 干涉模型：多光束干涉的数学描述

考虑光在外延层界面和衬底界面产生多次反射和透射，进行相干叠加的结果。

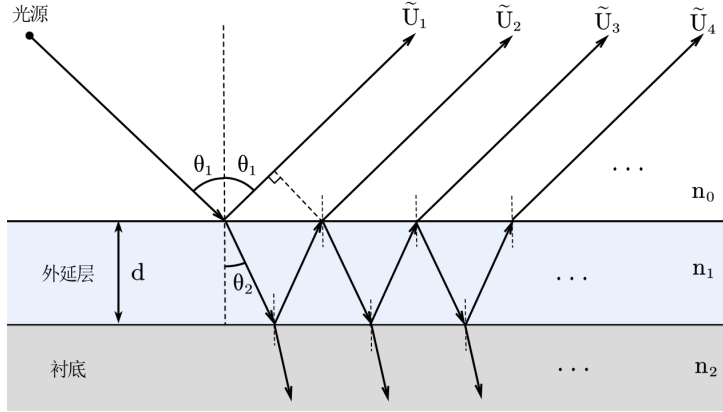


图 9 多光束干涉光路图

各级反射光复振幅表达式：

$$\begin{cases} \tilde{U}_1 = r_1 A_0, \\ \tilde{U}_2 = r_2 (t_1 t_1') e^{-i\delta} A_0, \\ \tilde{U}_3 = r_2^2 r_1' (t_1 t_1') e^{-i(2\delta)} A_0, \\ \tilde{U}_4 = r_2^3 r_1'^2 (t_1 t_1') e^{-i(3\delta)} A_0, \\ \vdots \end{cases}$$

对所有复振幅求和，并带入互易关系 $t_1 t_1' - r_1 r_1' = 1$ 得到：

$$\tilde{U}_R = \left(\frac{r_1 + r_2 e^{-i\delta}}{1 + r_1 r_2 e^{-i\delta}} \right) A_0 \quad (23)$$

对 s 光和 p 光分别计算：

$$\tilde{U}_s = \left(\frac{r_{1s} + r_{2s} e^{-i\delta}}{1 + r_{1s} r_{2s} e^{-i\delta}} \right) A_0 \quad (24)$$

$$\tilde{U}_p = \left(\frac{r_{1p} + r_{2p} e^{-i\delta}}{1 + r_{1p} r_{2p} e^{-i\delta}} \right) A_0 \quad (25)$$

二者非相干叠加，得到光强，计算总反射率：

$$R = \frac{\left| \frac{r_{1s} + r_{2s} e^{-i\delta}}{1 + r_{1s} r_{2s} e^{-i\delta}} \right|^2 + \left| \frac{r_{1p} + r_{2p} e^{-i\delta}}{1 + r_{1p} r_{2p} e^{-i\delta}} \right|^2}{2} \quad (26)$$

8.2 发生多光束干涉的必要条件

8.2.1 相干性条件

由于现实情况下光的相干长度有限，如果要发生多光束干涉，需要相干长度远大于在介质中往返一次的光程差，这样才能保证同一列光波能在经过多次反射后仍能 and 自身发生干涉。

$$L_c \gg 2nd \cos \theta \quad (27)$$

8.2.2 反射率与吸收条件

考虑第 k 次反射波的振幅和第 $k+1$ 次反射波振幅：

$$\tilde{U}_k = r_2^{k-1} r_1'^{k-2} (t_1 t_1') e^{-i(k-1)\delta} A_0 \quad (28)$$

$$\tilde{U}_{k+1} = r_2^k r_1'^{k-1} (t_1 t_1') e^{-i(k)\delta} A_0 \quad (29)$$

计算 $(k+1)$ 次反射光和 k 次反射光振幅比，得到：

$$\begin{cases} \left| \frac{\tilde{U}_1}{\tilde{U}_2} \right| = \left| \frac{r_2 t_1 t_1' e^{-i\delta}}{r_1} \right| & k = 1 \\ \left| \frac{\tilde{U}_{k+1}}{\tilde{U}_k} \right| = |r_1' r_2 e^{-i\delta}| & k \geq 2 \end{cases}$$

若要发生多光束干涉，需要满足经过多次透射折射后的反射光振幅衰减较慢。

在这里，我们认为如果：

$$|r_1' r_2 e^{-i\delta}| \sim 10^{-1} - 10^0 \quad (30)$$

则多次反射光对于最终光强仍有较大贡献，需要考虑多光束产生的影响。

若：

$$|r_1' r_2 e^{-i\delta}| \ll 1 \quad (31)$$

则多次反射波的振幅衰减极快，对光强的影响将小于数据噪音，可以忽略，只需考虑双光束干涉。

8.2.3 几何与均匀性条件

两界面基本平行且足够光滑，膜层厚度在光斑内均匀，这样相邻往返的相位差保持恒定值：

$$\delta = \frac{4\pi n d \cos \theta}{\lambda} \quad (32)$$

8.3 多光束干涉对厚度计算精度的影响分析

8.3.1 对比度分析

对比度 V 是衡量波峰和波谷光强变化相对于本底光强的明显程度的物理量，它的定义是：

$$V = \frac{I_{max} - I_{min}}{I_{max} + I_{min}} \quad (33)$$

通过波动图像的峰值推算厚度 d ，对比度越大，波峰和波谷越明显，对峰值处的波数值测量更准确。反之，如果对比度很小，峰值测量误差大，会导致最终算出的 d 存在较大误差。

对于双光束干涉，若两束光的强度分别为 I_1, I_2 ，二者相位差为 δ 则对比度由公式：

$$V_{2beam} = \frac{2\sqrt{I_1 I_2}}{I_1 + I_2} \quad (34)$$

对多光束干涉，若衬底和空气折射率差异不大，可近似使用经典 Airy 公式给出透射强度：

$$T(\delta) = \frac{1}{1 + F \sin^2\left(\frac{\delta}{2}\right)}, \quad F = \frac{4R}{(1 - R)^2}. \quad (35)$$

得到对比度为：

$$V_{multi} = \frac{F}{F + 2} \quad (36)$$

在通常情况下 $V_{multi} \gg V_{2beam}$ ，若发生多光束干涉，能使条纹对比度更大，测量和标定的峰值坐标更加准确，厚度计算精度更高。

8.3.2 精细度分析

光谱精细度 F 是衡量干涉条纹尖锐程度的物理量，由文献 [3]，精细度被定义为干涉条纹的自由光谱范围 (FSR, Free Spectral Range) 与半高全宽 (FWHM, Full Width at Half Maximum) 的比值：

$$F = \frac{FSR}{FWHM} = \frac{\Delta\sigma}{\delta\sigma} \quad (37)$$

光谱精细度 F 越大，波峰越锐利，对波峰处数值的定位和测量更加准确。对于双光束干涉，其干涉条纹呈余弦分布，由定义计算：

$$F_{2beam} \approx 2 \quad (38)$$

对多光束干涉，谱线会出现明显的锐利尖峰，在合适条件下 F 会远大于 2：

$$F_{multi} > 2 \quad (39)$$

若发生多光束干涉，其干涉条纹在频谱上更加锐利，对峰值的定位更加准确，厚度计算精度更高。

8.4 对硅晶圆片发生多光束干涉的定性判断

对于双光束干涉，可以得到峰值间隔 $\Delta\sigma \approx \frac{1}{2nd\cos\theta}$ ，将反射率 R 的表达式抽象为函数形式：

$$R(\sigma) = A_1(\sigma) + B_1(\sigma)\cos(C_1\sigma) \quad (40)$$

若发生多光束干涉，峰值之间的波数间隔 $\Delta\sigma \approx \frac{1}{2nd\cos\theta}$ 与双光束一致，对于 R 的表达式，可以抽象为函数形式：

$$R(\sigma) = 1 - \frac{A_2(\sigma)}{B_2(\sigma) + C_2(\sigma)\cos(D_2\sigma)} \quad (41)$$

为考察其波动特征，可以先忽略振幅调制，只考虑相位对反射率图像的影响，取合适参数作图：

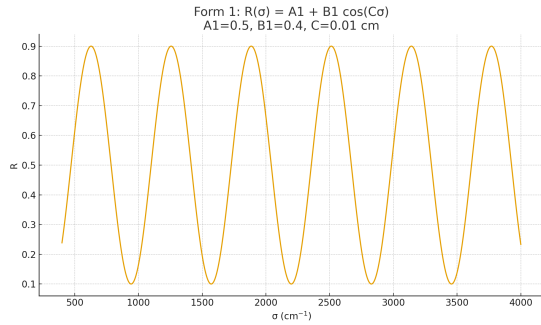


图 10 双光束干涉反射率形貌图

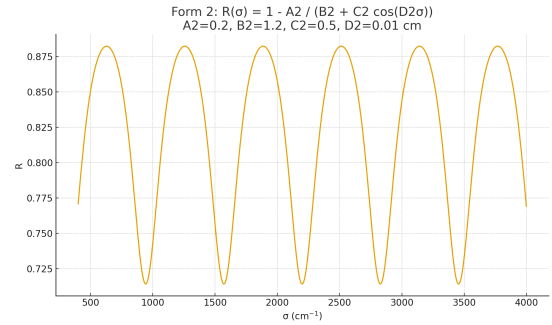


图 11 多光束干涉反射率形貌图

对比发现，双光束干涉的波峰和波谷基本对称。而多光束呈现出波峰较圆，波谷较尖的特征，与附件 3, 4 图像中出现的波峰波谷非对称现象相符。

据此，可以定性推断附件 3 和附件 4 提供的硅晶圆片的测试结果出现了多光束干涉。

8.5 基于干涉极值点的外延层厚度数学模型

在确认附件 3 和附件 4 的光谱具有明显的多光束干涉特征后，我们设计了一种基于干涉极值点（波峰与波谷）位置的解析算法来精确求解外延层厚度。

8.5.1 算法模型选择

与问题二不同，问题三的硅晶圆片产生的光谱是由高透明介质产生的**高保真多光束干涉条纹**，所以其光学响应主要由实部折射率 n 确定。这意味着光谱中反射率仅与厚度 d 存在函数关系，与其他物理参数基本无关。基于此，我们选择**干涉极值点解析法**。

8.5.2 折射率模型

硅与碳化硅在晶体结构上存在根本差异，其晶格振动在红外波段高度透明，所以我们无法直接沿用问题二的折射率模型。通过使用 AI[1] 进行文献搜索，我们发现对于硅外延层，其在红外波段的折射率色散关系可以通过一个三项的塞尔迈耶方程精确描述。根据文献 [6]，该方程形式如下：

$$n_{\text{Si}}^2(\lambda) = 1 + \sum_{j=1}^3 \frac{B_j \lambda^2}{\lambda^2 - C_j} \quad (42)$$

其中， $n_{\text{Si}}(\lambda)$ 是硅在波长 λ 处的折射率。通过阅读文献，我们采用色散系数 B_j 和 C_j 这一组经验常数，其数值如下：

$$\begin{aligned} B_1 &= 10.6684293, & C_1 &= 0.09091219 \mu\text{m}^2 \\ B_2 &= 0.0030434748, & C_2 &= 1.28766018 \mu\text{m}^2 \\ B_3 &= 1.54133408, & C_3 &= 1218816 \mu\text{m}^2 \end{aligned}$$

8.5.3 干涉模型

干涉光谱的极值点（波峰与波谷）满足其相位差 δ 为 π 的整数倍。我们可以将此条件统一表示为 $\delta = k \cdot 2\pi$ (k 取 0.5 的整数倍)，其中，干涉级数因子 k 对于相长干涉的波峰是整数，对于相消干涉的波谷是半整数。

将相位差的表达式 $\delta = 4\pi n d \cos \theta_1 \sigma$ 代入上式，即可反解出厚度 d 的计算公式：

$$d = \frac{k}{2n(\sigma_k) \cos \theta_1 \sigma_k} \quad (43)$$

其中， σ_k 是对应级数因子为 k 的极值点波数， $n(\sigma_k)$ 是在该波数下由塞尔迈耶方程计算出的折射率。

8.5.4 算法流程与数据

1. **数据提取**: 从实验光谱数据中精确提取所有干涉极值点（波峰与波谷）的波数位置 σ_k 。表 4 列出了用于计算的精确数据。
2. **遍历搜索**: 设定一个合理的整数范围，遍历尝试第一个极值点的初始干涉级数 m_0 因子。
3. **厚度计算**: 对于每一个假定的 m_0 ，根据所有极值点的相对顺序确定它们的干涉级数序列（例如 $m_0, m_0 + 0.5, m_0 + 1, \dots$ ），并利用公式 (43) 计算出一组对应的厚度值 $\{d_k\}$ 。
4. **确定最优解**: 计算该组厚度值 $\{d_k\}$ 的标准差 $\text{std}(\{d_k\})$ 。能够使标准差 $\text{std}(\{d_k\})$ 达到最小的 m_0 即为正确的初始干涉级数。该 m_0 对应的厚度平均值 $\bar{d}$ 即为最终求解结果。

表 4 附件 3 与附件 4 光谱的精确极值点数据

(a) 附件 3 (入射角 10°)			(b) 附件 4 (入射角 15°)		
序号	波数 (cm ⁻¹)	类型	序号	波数 (cm ⁻¹)	类型
1	750.17	波峰	1	574.20	波谷
2	940.61	波谷	2	749.21	波峰
3	1105.49	波峰	3	946.40	波谷
4	1312.80	波谷	4	1108.39	波峰
5	1508.06	波峰	5	1324.38	波谷
6	1722.60	波谷	6	1519.15	波峰
7	1927.02	波峰	7	1736.10	波谷
8	2144.94	波谷	8	1941.97	波峰

基于极值点法的厚度一致性分析与最优干涉级数确定

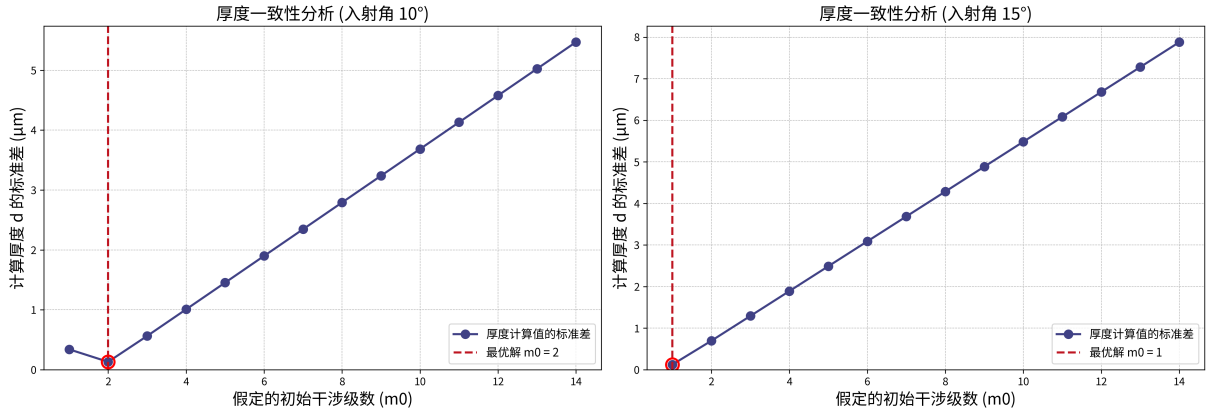


图 12 干涉极值法得出的厚度一致性与最优干涉级数确定图

8.5.5 计算结果与分析

干涉极值法得出的结果如图 (12)，对于附件 3，算法在初始干涉级数 $m_0 = 2$ 时取得最小标准差，计算得到的外延层厚度为 $d_3 = 3.7504 \mu\text{m}$ 。对于附件 4，算法在初始干涉级数 $m_0 = 1$ 时取得最小标准差，计算得到的外延层厚度为 $d_4 = 3.7520 \mu\text{m}$ ，相对误差约为 0.04%，可判断最终结果约为 $d = 3.75 \mu\text{m}$ 。

8.6 测试结果发生多光束干涉的证明

8.6.1 相干性条件与几何均匀性条件：

反射率函数有显著的波动性，利用多光束波动模型计算得到的结果稳定高，且图像形貌满足多光束干涉现象，判断满足相干性条件和几何均匀性条件。

8.6.2 反射率与吸收条件：

在计算得到外延层厚度后，可通过反射率实验数值与菲涅尔公式，反解出 n_2 随波数的变化关系图。将 n_2 的结果反带回菲涅尔公式，可以算出直接反射光相对振幅大小 $|\tilde{U}_1|$ 、经过透射后一次反射光 $|\tilde{U}_2|$ 等，据此作出前五个反射光相对振幅大小随波数变化图：

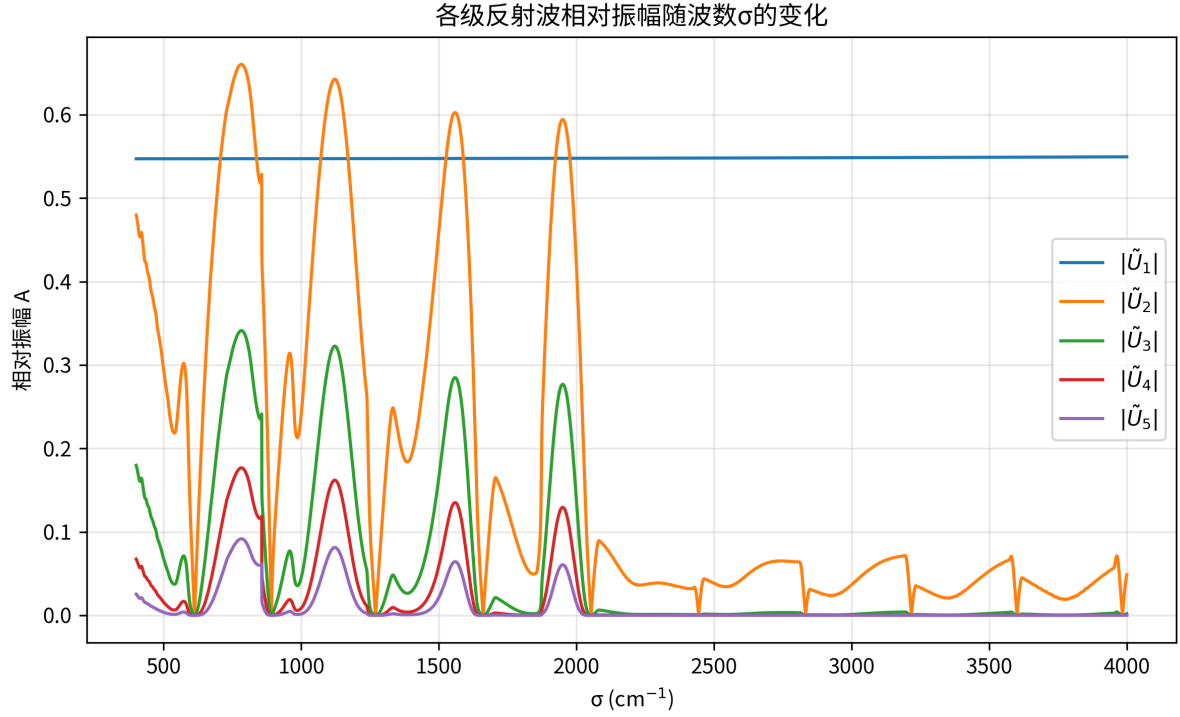


图 13 各级反射光相对光强随波数变化图像

根据图像可以发现，在波数位于 $400\text{-}2000\text{cm}^{-1}$ 区间，各级反射光光振幅较大，多光束干涉效应明显，经过估算得到：

$$\left| \frac{\tilde{U}_{k+1}}{\tilde{U}_k} \right| = |r'_1 r_2 e^{-i\delta}| \approx 0.5 \quad (44)$$

满足发生多光束干涉的判定条件。

8.6.3 精细度判定条件:

将数据代入公式 (37) 计算，附件 3、4 的实验光谱对应的精细度分别为 2.7079 和 2.6740，大于 2，由此可判断硅晶圆片产生了多光束干涉。

8.7 基于色散矫正的傅里叶变换的外延层厚度数学模型

在对数据的处理过程中，我们发现存在信噪比较低和干涉条纹不清晰的现象。并由图 13 观察发现，当波数大于 2000cm^{-1} 时，经过多级反射的光振幅明显减弱，多光束干涉效应不再明显，导致难以准确定位极值点。

为了充分利用实验数据，我们和 AI[3] 讨论并引入了基于快速傅里叶变换 (FFT) 的分析方法来从信号中提取频率信息，计算出外延层厚度。其中，我们创新性地通过坐标变换解决了硅的色散导致干涉信号在波数域内并非严格周期性的问题。

8.7.1 算法核心思想与色散问题

我们首先回顾干涉的相位差公式：

$$\delta(\sigma) = 4\pi d \cdot \sigma \cdot n_f(\sigma) \cos(\theta_j) \quad (45)$$

其中, σ 是波数, d 是外延层厚度, $n_f(\sigma)$ 是外延层随波数变化的折射率, θ_i 是光在空气中的入射角, θ_j 是光在外延层中的折射角。

利用斯涅尔公式, 并考虑到本问折射率为实数:

$$\delta(\sigma) = 4\pi d \cdot \sigma \cdot \sqrt{n_f(\sigma)^2 - \sin^2(\theta_i)} \quad (46)$$

若直接对以波数 σ 为自变量的反射光谱进行傅里叶变换, 会遇到问题: 由于折射率 $n_f(\sigma)$ 的存在, 相位差 δ 与波数 σ 之间并非严格的线性关系。这意味着直接对其进行 FFT 会导致频谱中的峰变宽并产生位置偏移, 从而引入系统误差。

8.7.2 坐标变换: 线性化相位

为了解决此问题, 我们决定在进行 FFT 之前, 对数据进行一次坐标变换, 将其从波数域转换到一个新的坐标域, 在这个新域里, 干涉信号具有周期性。根据相位公式, 我们定义一个新的坐标变量 x :

$$x(\sigma) = 2\sigma \sqrt{n_f(\sigma)^2 - \sin^2(\theta_i)} \quad (47)$$

将此新坐标代入相位公式, 我们得到:

$$\delta(x) = 2\pi d \cdot x \quad (48)$$

在这个新的 x 域中, 相位差与坐标 x 呈现较好的线性关系, 所以反射光谱 $R(x)$ 是一个真正意义上的周期信号, 通过对信号进行建模分析可以计算得到厚度 d 。

8.7.3 算法流程

我们和 AI[1] 优化了算法细节, 如增加了汉宁窗函数的应用。最终算法流程如下:

1. **数据预处理与坐标变换:** 我们手动标记附件 3、4 数据的波峰波谷位置 (σ_i, R_i) , 利用公式 (47) 和公式 (42), 计算出每个原始波数点 σ_i 所对应的非均匀坐标点 x_i 。为了进行 FFT, 我们必须将光谱数据转化为具有周期性的函数, 因此我们通过线性插值法, 将反射率数据 R 重新采样到一个均匀间隔的新坐标网格 $x_{uniform}$ 上。为提高分辨率, 我们选取 16384 为采样点数。
2. **窗函数应用:** 由于我们处理的是有限长度的信号, 直接进行 FFT 会导致信号的两端不连续, 干扰主峰的识别。所以我们在 FFT 之前, 将重采样后的信号乘了一个平滑的窗函数, 我们选用汉宁窗。为使 FFT 频谱中的主峰更加突出, 我们从信号中减去了其直流分量 (均值)。
3. **快速傅里叶变换 (FFT):** 对经过加窗处理的信号应用标准的 FFT 算法。由于我们已经进行了精确的坐标变换, 得到的 FFT 频谱的横坐标直接对应于薄膜的物理厚度 d 。
4. **高精度峰定位:** FFT 的结果是离散的, 其峰值位置被限制在频率栅格上。为了获得“亚像素”级别的精度, 我们首先在 FFT 振幅谱中找到最大值点, 然后选取该点及其左右两个相邻点, 通过一个二次多项式 (抛物线) 进行拟合, 并解析地计算出该抛物线的顶点位置。这个顶点位置就是我们最终得到的高精度厚度值。

8.7.4 计算结果与交叉验证

我们将此算法应用于附件 3 和附件 4 的数据, 我们得到了相似的结果, 如图 (14)。前者结果为 $d_3 = 3.6070 \mu\text{m}$, 后者为 $d_4 = 3.5833 \mu\text{m}$ 。

8.8 外延层厚度最终结论

经过以上对计算精度的分析，我们最终选择基于傅里叶变换 (FFT) 的方法求解出来的结果，也就是对于附件 3 (10°)，厚度为 $d_3 = 3.6070 \mu\text{m}$ 。对于附件 4 (15°)，厚度为 $d_4 = 3.5833 \mu\text{m}$ ，结果在误差范围内，我们取两次计算结果的平均值 3.5952 m 作为最终报告厚度。

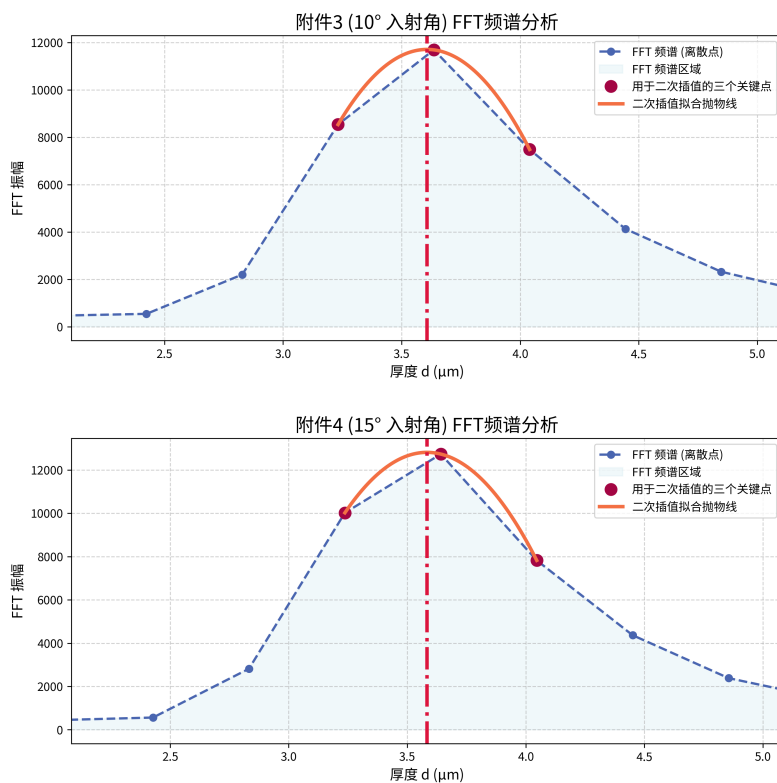


图 14 对于入射角 10° 和 15° 的 FFT 频谱分析与厚度求解

9 碳化硅晶圆片的测试结果无需考虑多光束干涉的说明

利用在处理问题二得到的折射率随波数变化图像，可以利用菲涅尔公式计算出各级反射振幅的大小，画出振幅图如图 (15)。

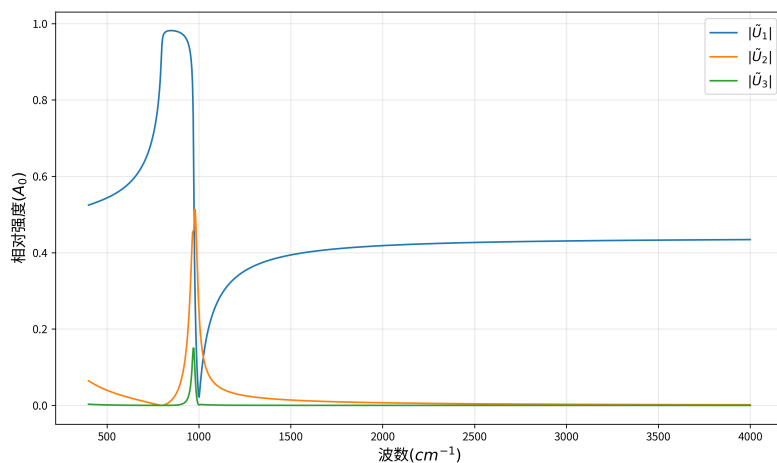


图 15 问题二各级反射光相对强度图

根据图像，可以发现光强的主要贡献项为直接反射光。多级反射光光强远小于直接反射光和一次反射光。即：

$$|r_1' r_2 e^{-i\delta}| \ll 1 \quad (49)$$

所以无需考虑多光束干涉对之前计算结果的影响。

10 模型检验

10.1 问题三基于傅里叶变换的计算外延层厚度方法灵敏度

首先，我们分别对入射角 θ_i 和塞尔迈耶方程中的主导色散系数 B_1 的基准值加入了 $\pm 2\%$ 和 $\pm 1\%$ 的扰动。同时，我们还对 FFT 前的重采样点数 `num_points` 进行了收敛性分析，以证明我们选取参数的合理性。图 (16) 展示了这两项灵敏度分析的结果。

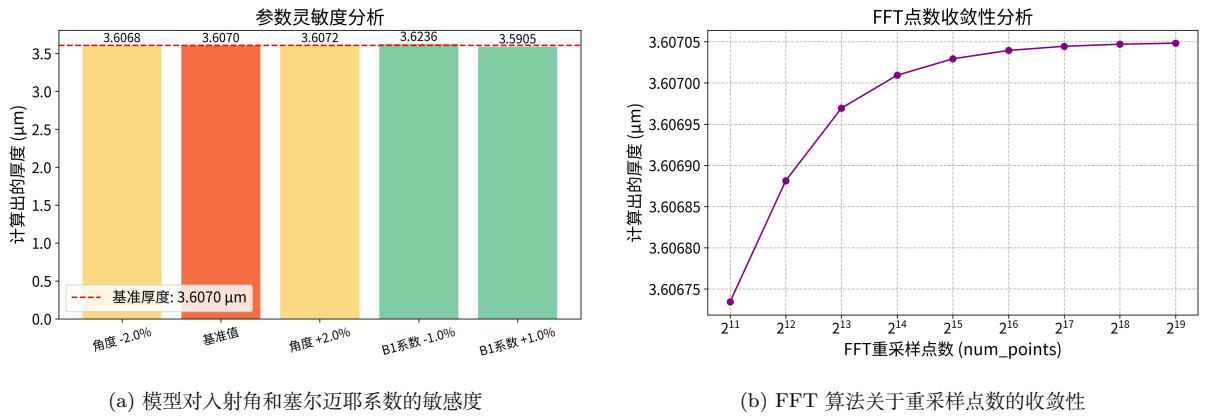


图 16 FFT 算法的灵敏度与收敛性综合分析

从图 (16a) 可看出，入射角仅引起厚度计算结果约 0.005% 的微小波动，而 B_1 系数引起的波动也小于 0.5%。这证明了该算法在面对实际测量误差和材料光学常数不确定性时具有出色的稳健性。

同时，从图 (16b) 可以看到，当采样点数超过 8192 (2^{13}) 时，计算结果已收敛到一个稳定的数值，进一步增加点数对结果的精度的提升已经非常小。这证明了我们在算法中所采用的点数 ($2^{14}=16384$) 有效。

10.2 问题二、三外延层厚度算法稳定性检验

10.2.1 问题二基于双光束干涉和洛伦兹-德鲁德模型计算外延层厚度方法稳定性

我们对附件 1 和 2 的原始反射率数据都人为地叠加了标准差为 0.5% 的高斯白噪声，并且重复此过程 100 次，每次都使用带有不同随机噪声序列的数据，进行厚度变化率计算。

由结果统计直方图 (17) 可知，所有计算结果的均值变化率 $< 0.0001\%$ ，说明我们建立的基于双光束干涉和洛伦兹-德鲁德模型计算外延层厚度模型非常有效，能够对含有真实测量噪声的光谱数据进行精确分析。

10.2.2 问题三基于傅里叶变换的计算外延层厚度方法稳定性

类似于问题二的检验，我们对附件 3 的原始反射率数据叠加了标准差为 0.5% 的高斯白噪声，重复 200 次，每次都使用带有不同随机噪声序列的数据，进行厚度计算。计算结果如图 (18) 所示。

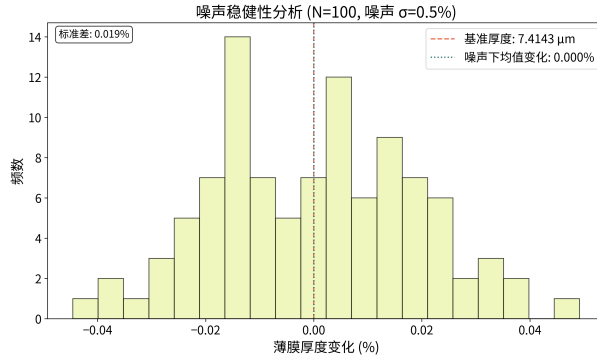


图 17 FFT 算法在随机噪声下的稳定性检验（蒙特卡洛模拟结果）

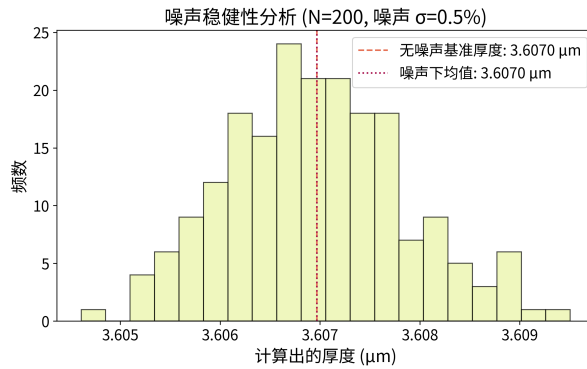


图 18 基于双光束干涉和洛伦兹-德鲁德模型计算外延层厚度方法在随机噪声下的稳定性检验（蒙特卡洛模拟结果）

我们发现，所有计算值的均值（ $3.6070 \mu\text{m}$ ，图中紫色虚线所示）与无噪声情况下的基准真值（ $3.6070 \mu\text{m}$ ，图中红色虚线所示）相近。这充分表明，我们所建立的基于傅里叶变换的算法具有较好的噪声抑制能力。

11 模型评价

11.1 模型的优点

- 物理模型解释性强：**本文针对碳化硅和硅的不同材料特征，分别建立了基于洛伦兹-德鲁德模型和三项塞尔迈耶方程的物理模型，解释了光谱形成的物理机制，具有很高精度与实际价值。
- 算法策略精度高：**针对问题二，我们采用了“全局同步拟合”策略和“差分进化 + 局部优化”的先进算法，有效避免了局部最优解，保证了结果的全局最优性。针对问题三，我们创新性地引入了带色散校正的傅里叶变换法，通过坐标变换有效地解决了 FFT 的系统误差问题。
- 结果可靠性高：**本文不仅给出了计算结果，还通过多种手段对其可靠性进行了交叉验证。例如，通过计算折射率验证了双光束模型的合理性；通过对比不同角度的计算结果验证了一致性；通过灵敏度和稳定性分析（蒙特卡洛模拟）全面检验了模型的鲁棒性。

11.2 模型的缺点

- 标准洛伦兹-德鲁德模型对高掺杂衬底的描述不佳：**在问题二中，残差分析显示，模型在“剩余射线带”存在系统性偏差。这反映，标准的洛伦兹-德鲁德模型可能未能完全捕捉到高度掺杂半

导体中声子与等离激元之间的复杂耦合效应。

2. **对碳化硅材质的假设局限性：**问题二中假设碳化硅是标准的 4H-SiC 材料，而实际上工业生产的碳化硅晶圆可能存在多种晶型，例如 6H-SiC 或 3C-SiC，它们具有不同的晶格结构，这将直接导致其本征光学参数不同。此假设引入了系统性误差。
3. **复杂拟合中的参数耦合风险：**在问题二的七参数全局拟合的逆问题中，存在参数耦合的固有挑战——理论上可能存在另一组略有不同的参数组合，也能产生非常相似的拟合光谱。
4. **对固定参数精确性的依赖：**问题二的固定洛伦兹-德鲁德模型参数，以及问题三的两项塞尔迈耶方程参数，其精度都高度依赖于文献所给参数的准确性。文献中的标准值可能与实际样品的真实光学常数存在微小差异。

11.3 模型的改进

1. **引入声子-等离激元耦合模型代替标准洛伦兹-德鲁德模型：**根据文献 [5]，在高载流子浓度的碳化硅中，声子和等离激元会发生强烈的耦合。我们可以采用文献中一个统一的 **LO 声子-等离激元耦合模型**来描述介电函数，来减少在“剩余射线带”观察到的系统性拟合偏差。
2. **拉曼光谱法进行晶型无损鉴定：**为了解决碳化硅晶型不确定导致系统误差的问题，我们可以先对样品进行拉曼光谱无损检测，鉴定出样品准确晶型，再为该晶型选取对应的正确光学本征参数代入红外光谱模型。
3. **采用贝叶斯推断优化算法：**为了解决复杂拟合中解的不唯一性，可以采用基于**马尔可夫链蒙特卡洛**的贝叶斯分析框架 [7]，此算法能够探索整个参数空间，并给出每个参数的后验概率分布。

参考文献

- [1] Gemini, 2.5 Pro (Release 2025-06-17), Google, 使用日期: 2025-09-04 至 2025-09-07[Z].
- [2] OISHI S, HIJIKATA Y, YAGUCHI H, et al. Simultaneous determination of carrier concentration, mobility, and thickness of SiC homoepilayers by infrared reflectance spectroscopy[J]. Japanese journal of applied physics, 2006, 45(12L): L1226.
- [3] Claude, 4 Sonnet (Release 2025-05-22), Anthropic, 使用日期: 2025-09-04 至 2025-09-07[Z].
- [4] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek), 使用日期: 2025-09-05 至 2025-09-07[Z].
- [5] SHOKHOVETS S, GOBSCH G, GOLDHAHN R. Infrared reflectance and ellipsometry study of LO phonon-plasmon coupling in n-type 4H-SiC[J]. Journal of Applied Physics, 2011, 110(3): 033534.
- [6] LI H. Refractive index of silicon and germanium and its wavelength and temperature derivatives [J]. Journal of Physical and Chemical Reference Data, 1980, 9(3): 561-658.
- [7] ARREGUI J A, DE BUSTOS C, GARCA-CAUREL E. Bayesian inference for the analysis of spectroscopic ellipsometry data[J]. Optics Express, 2019, 27(20): 29013-29029.

A 支撑材料文件列表

- q2.py: 用于问题二模型求解的核心 Python 代码。
- q3-interfere.py: 用于问题三基于干涉极值点求解的核心 Python 代码。
- q3-fft.py: 用于问题三基于色散矫正傅里叶变换求解的核心 Python 代码。
- q2-verification: 问题二模型稳定性检验。
- sensitivity.py: 用于检验问题二、三模型的灵敏度。
- AI 工具使用详情说明.pdf: AI 工具使用详情说明。

B 主要源程序代码

B.1 问题二求解的 Python 关键代码 (q2.py)

```
1 # -*- coding: utf-8 -*-
2 import warnings
3
4 warnings.filterwarnings("ignore")
5
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 from scipy.optimize import minimize, differential_evolution
10 import zhplot
11 from copy import deepcopy
12
13 # ===== 常量与入射条件 =====
14 eps_inf = 6.56
15 theta1 = 15 / 180 * np.pi # 入射角 (弧度)
16 n0 = 1.0 # 入射介质 (空气)
17
18 # ===== 赝等离激元-晶格 (Lorentz + Drude) 介电函数 =====
19 # 所有频率量纲均为  $\text{cm}^{-1}$ ; sigma 也是  $\text{cm}^{-1}$ 
20 omega_L = 970.0
21 omega_T = 798.0
22
23
24 def get_epsilon(sigma, gamma_L, gamma_T, omega_p, gamma_p):
25     """
26     sigma:  $\text{cm}^{-1}$  (可为标量或 ndarray)
27     其余参数:  $\text{cm}^{-1}$ 
28     返回: 复介电常数 ( )
29     """
30     sigma = np.asarray(sigma, dtype=np.complex128)
31     lattice = eps_inf * (
32         (omega_L**2 - sigma**2 - 1j * gamma_L * sigma)
33         / (omega_T**2 - sigma**2 - 1j * gamma_T * sigma)
34     )
35     drude = (omega_p**2) / (sigma**2 + 1j * gamma_p * sigma)
36     return lattice - drude
37
38
39 def fresnel_rt_and_theta2(n_i, n_j, theta_i):
```

```

40 """
41 允许 n_j 为复数/数组；返回 (r_s, r_p, t_s, t_p, theta_j)
42 """
43 n_i = np.asarray(n_i, dtype=np.complex128)
44 n_j = np.asarray(n_j, dtype=np.complex128)
45 theta_i = np.asarray(theta_i, dtype=np.complex128)
46
47 sin_theta_j = (n_i / n_j) * np.sin(theta_i)
48 theta_j = np.arcsin(sin_theta_j)
49
50 cos_i = np.cos(theta_i)
51 cos_j = np.cos(theta_j)
52
53 r_s = (n_i * cos_i - n_j * cos_j) / (n_i * cos_i + n_j * cos_j)
54 t_s = (2 * n_i * cos_i) / (n_i * cos_i + n_j * cos_j)
55
56 r_p = (n_j * cos_i - n_i * cos_j) / (n_j * cos_i + n_i * cos_j)
57 t_p = (2 * n_i * cos_i) / (n_j * cos_i + n_i * cos_j)
58
59 return r_s, r_p, t_s, t_p, theta_j
60
61
62 def get_I(sigma, d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2):
63 """
64 双光束近似：首反射 + 一次往返
65 返回 非偏振反射率 R( ) (0-1)
66 """
67 sigma = np.asarray(sigma, dtype=np.complex128)
68 n1 = np.sqrt(get_epsilon(sigma, gamma_L, gamma_T, omega_p1, gamma_p1)) # 薄膜
69 n2 = np.sqrt(get_epsilon(sigma, gamma_L, gamma_T, omega_p2, gamma_p2)) # 衬底
70
71 r01s, r01p, t01s, t01p, theta2 = fresnel_rt_and_theta2(n0, n1, theta1)
72 r12s, r12p, t12s, t12p, _ = fresnel_rt_and_theta2(n1, n2, theta2)
73 r10s, r10p, t10s, t10p, _ = fresnel_rt_and_theta2(n1, n0, theta2)
74
75 d_cm = d_um * 1e-4
76 delta = 4 * np.pi * sigma * d_cm * n1 * np.cos(theta2)
77
78 A_s = r01s + t01s * r12s * t10s * np.exp(1j * delta)
79 A_p = r01p + t01p * r12p * t10p * np.exp(1j * delta)
80
81 R = (np.abs(A_s) ** 2 + np.abs(A_p) ** 2) / 2.0
82 return np.real_if_close(R)
83
84
85 # ===== 数据读取 =====
86 def load_data(excel_path):
87 """
88 读取单个xlsx文件并返回处理后的数据
89 返回：(sigma_data, y_data)
90 """
91 # 读取（含表头），两列：[sigma(cm^-1), 目标值(百分数)]
92 df_raw = pd.read_excel(excel_path)
93
94 # 尝试自动识别两列（容错：列名未知）
95 # 将前两列转换为数值；非数值转为 NaN
96 df = df_raw.iloc[:, :2].copy()
97 df.columns = ["sigma", "y_percent"]
98 df["sigma"] = pd.to_numeric(df["sigma"], errors="coerce")

```

```

99     df["y_percent"] = pd.to_numeric(df["y_percent"], errors="coerce")
100
101     # 丢弃无效行
102     df = df.dropna(subset=["sigma", "y_percent"]).reset_index(drop=True)
103
104     # 从"第二个数据点开始读取" → 跳过第一行有效数据
105     if len(df) >= 2:
106         df = df.iloc[1:].reset_index(drop=True)
107
108     sigma_data = df["sigma"].to_numpy()
109     y_data = df["y_percent"].to_numpy() # 百分数形式 (例如 30 表示 30%)
110
111     return sigma_data, y_data
112
113
114 # 读取两个Excel文件
115 excel_path1 = r"./attach/1.xlsx"
116 excel_path2 = r"./attach/2.xlsx"
117
118 sigma_data1, y_data1 = load_data(excel_path1)
119 sigma_data2, y_data2 = load_data(excel_path2)
120
121 print(
122     f"数据集1: {len(sigma_data1)} 个数据点, 波数范围: {sigma_data1.min():.1f} - {sigma_data1.max():.1f} cm-1"
123 )
124 print(
125     f"数据集2: {len(sigma_data2)} 个数据点, 波数范围: {sigma_data2.min():.1f} - {sigma_data2.max():.1f} cm-1"
126 )
127
128
129 # ===== 目标函数 (最小化残差平方和) =====
130 def model_percent(sigma, params):
131     d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2 = params
132     R = get_I(sigma, d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2)
133     return 100.0 * R # 拟合目标是 R*100 (百分数)
134
135
136 def loss_single(sigma_data, y_data, params):
137     """计算单个数据集的loss"""
138     y_hat = model_percent(sigma_data, params)
139     # 对 NaN/Inf 做保护
140     if not np.all(np.isfinite(y_hat)):
141         return 1e12 # 返回一个非常大的值作为惩罚
142     resid = y_hat - y_data
143     return float(np.mean(resid**2))
144
145
146 def loss(params):
147     """计算双数据集的总loss (等权重)"""
148     loss1 = loss_single(sigma_data1, y_data1, params)
149     loss2 = loss_single(sigma_data2, y_data2, params)
150
151     # 如果任何一个数据集返回惩罚值, 则返回惩罚值
152     if loss1 >= 1e12 or loss2 >= 1e12:
153         return 1e12
154
155     # 等权重平均

```

```

156     return (loss1 + loss2) / 2.0
157
158
159 # =====
160 # 关键修改：为 differential_evolution 创建一个可被 pickle 的函数
161 # 这个函数在顶层定义，并直接调用现有的 'loss' 函数
162 def de_loss_wrapper(params):
163     return loss(params)
164
165
166 # =====
167
168 # ===== 初值与边界 =====
169 # 你提供的近似初值：
170 x0 = np.array([6.4, 6.0, 6.0, 65.0, 55.0, 480.0, 420.0], dtype=float)
171
172 if __name__ == "__main__":
173     # 边界（可根据你的样品实际情况再收紧/放宽）
174     bounds = [
175         (0.01, 100.0), # d_um
176         (0.1, 2000.0), # gamma_L
177         (0.1, 2000.0), # gamma_T
178         (1.0, 5000.0), # omega_p1
179         (0.1, 2000.0), # gamma_p1
180         (1.0, 5000.0), # omega_p2
181         (0.1, 2000.0), # gamma_p2
182     ]
183
184     # ===== 全局 + 局部 拟合 =====
185     # 1) 全局搜索
186     result_de = differential_evolution(
187         func=de_loss_wrapper, # <--- 这里修改了，使用 de_loss_wrapper
188         bounds=bounds,
189         strategy="best2bin",
190         maxiter=300,
191         popsize=40,
192         tol=1e-2,
193         polish=False,
194         updating="deferred",
195         workers=-1, # 建议先从 workers=1 开始测试，确保一切正常
196         # 如果需要加速，再改为 -1。
197         # 此时，由于 de_loss_wrapper 是 def 定义的，应该不会有 pickle 错误。
198         seed=42,
199         disp=True,
200     )
201
202     # 2) 以全局结果为起点做局部细化
203     result_local = minimize(
204         fun=loss, # minimize 函数没有多进程问题，可以直接用 loss 函数
205         x0=result_de.x,
206         method="L-BFGS-B",
207         bounds=bounds,
208         options=dict(maxiter=500, ftol=1e-12, gtol=1e-12),
209     )
210     best_params = result_local.x
211     best_mse = result_local.fun
212
213     param_names = [
214         "d_um",

```

```

215     "gamma_L",
216     "gamma_T",
217     "omega_p1",
218     "gamma_p1",
219     "omega_p2",
220     "gamma_p2",
221 ]
222 print("===== 拟合完成 =====")
223 for n, v in zip(param_names, best_params):
224     print(f"{n:>9s} = {v:.6g}")
225 print(f"总MSE (percent^2) = {best_mse:.6g}")
226
227 # 计算各数据集的单独指标
228 loss1 = loss_single(sigma_data1, y_data1, best_params)
229 loss2 = loss_single(sigma_data2, y_data2, best_params)
230 print(f"数据集1 MSE = {loss1:.6g}")
231 print(f"数据集2 MSE = {loss2:.6g}")
232
233 # 计算R²值
234 def calculate_r2(y_true, y_pred):
235     ss_res = np.sum((y_true - y_pred) ** 2)
236     ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
237     return 1 - (ss_res / ss_tot)
238
239 y_fit1 = model_percent(sigma_data1, best_params)
240 y_fit2 = model_percent(sigma_data2, best_params)
241 r2_1 = calculate_r2(y_data1, y_fit1)
242 r2_2 = calculate_r2(y_data2, y_fit2)
243 print(f"数据集1 R² = {r2_1:.6f}")
244 print(f"数据集2 R² = {r2_2:.6f}")
245
246 # ===== 可视化 =====
247 # 计算残差
248 resid1 = y_fit1 - y_data1
249 resid2 = y_fit2 - y_data2
250
251 # 数据集1的拟合图
252 plt.figure(figsize=(10, 6))
253 plt.plot(
254     sigma_data1, y_data1, "o", color="#a30543", ms=5, label="实验数据", alpha=0.7
255 )
256 plt.plot(sigma_data1, y_fit1, color="#297270", lw=3, label="拟合曲线")
257 plt.gca().invert_xaxis() # 红外谱常用：高到低
258 plt.xlabel("波数 (cm⁻¹)", fontsize=16)
259 plt.ylabel("反射率 (%)", fontsize=16)
260 plt.title(f"附件1拟合结果 (R² = {r2_1:.4f})", fontsize=18)
261 plt.legend(fontsize=14)
262 plt.grid(True, alpha=0.3)
263 plt.tick_params(labelsize=14)
264 plt.tight_layout()
265 plt.savefig("数据集1_拟合结果.png", dpi=300, bbox_inches="tight")
266 plt.show()
267
268 # 数据集2的拟合图
269 plt.figure(figsize=(10, 6))
270 plt.plot(
271     sigma_data2, y_data2, "o", color="#a30543", ms=5, label="实验数据", alpha=0.7
272 )
273 plt.plot(sigma_data2, y_fit2, "#297270", lw=3, label="拟合曲线")

```

```

274 plt.gca().invert_xaxis() # 红外谱常用：高到低
275 plt.xlabel("波数 (cm$^{-1}$)", fontsize=16)
276 plt.ylabel("反射率 (%)", fontsize=16)
277 plt.title(f"附件2拟合结果 (R2 = {r2_2:.4f})", fontsize=18)
278 plt.legend(fontsize=14)
279 plt.grid(True, alpha=0.3)
280 plt.tick_params(labelsize=14)
281 plt.tight_layout()
282 plt.savefig("数据集2_拟合结果.png", dpi=300, bbox_inches="tight")
283 plt.show()
284
285 # 数据集1的残差图
286 plt.figure(figsize=(10, 4))
287 plt.plot(sigma_data1, resid1, "o", color="#a30543", ms=4, alpha=0.7)
288 plt.axhline(0, lw=2, color="#297270", linestyle="--")
289 plt.gca().invert_xaxis()
290 plt.xlabel("波数 (cm$^{-1}$)", fontsize=16)
291 plt.ylabel("残差 (%)", fontsize=16)
292 plt.title("附件1残差分析", fontsize=18)
293 plt.grid(True, alpha=0.3)
294 plt.tick_params(labelsize=14)
295 plt.tight_layout()
296 plt.savefig("数据集1_残差分析.png", dpi=300, bbox_inches="tight")
297 plt.show()
298
299 # 数据集2的残差图
300 plt.figure(figsize=(10, 4))
301 plt.plot(sigma_data2, resid2, "o", color="#a30543", ms=4, alpha=0.7)
302 plt.axhline(0, lw=2, color="#297270", linestyle="--")
303 plt.gca().invert_xaxis()
304 plt.xlabel("波数 (cm$^{-1}$)", fontsize=16)
305 plt.ylabel("残差 (%)", fontsize=16)
306 plt.title("附件2残差分析", fontsize=18)
307 plt.grid(True, alpha=0.3)
308 plt.tick_params(labelsize=14)
309 plt.tight_layout()
310 plt.savefig("数据集2_残差分析.png", dpi=300, bbox_inches="tight")
311 plt.show()
312
313 # ===== 复介电常数图 =====
314 # 创建波数范围用于绘制复介电常数
315 sigma_range = np.linspace(500, 3900, 3400)
316
317 # 使用拟合得到的参数计算复介电常数
318 epsilon_film = get_epsilon(sigma_range, best_params[1], best_params[2], best_params[3],
319                             best_params[4])
319 epsilon_substrate = get_epsilon(sigma_range, best_params[1], best_params[2], best_params[5],
320                                 best_params[6])
321
322 # 计算复折射率 n = sqrt(epsilon), 然后取实部
322 n_film = np.sqrt(epsilon_film).real
323 n_substrate = np.sqrt(epsilon_substrate).real
324
325 # 绘制折射率实部随波数的变化
326 plt.figure(figsize=(10, 6))
327 plt.plot(sigma_range, n_film, color="#297270", lw=3, label="外延层 n")
328 plt.plot(sigma_range, n_substrate, color="#a30543", lw=3, label="衬底层 n")
329 plt.gca().invert_xaxis()
330 plt.xlabel("波数 (cm$^{-1}$)", fontsize=16)

```

```

331 plt.ylabel("折射率 n", fontsize=16)
332 plt.title("折射率随波数变化", fontsize=18)
333 plt.legend(fontsize=14)
334 plt.grid(True, alpha=0.3)
335 plt.tick_params(labelsize=14)
336 plt.tight_layout()
337 plt.savefig("折射率_波数依赖性.png", dpi=300, bbox_inches="tight")
338 plt.show()
339
340 # ===== 灵敏度分析 =====
341 # 保存基准厚度
342 baseline_thickness = best_params[0]
343
344 # 1. 参数灵敏度分析 (eps_inf, omega_L, omega_T)
345 print("\n--- 正在进行参数灵敏度分析 ---")
346
347 # 参数扰动幅度
348 variation = 0.02 # ±2%
349
350 # 创建修改参数的函数
351 def get_epsilon_modified(sigma, gamma_L, gamma_T, omega_p, gamma_p, eps_inf_mod=eps_inf,
352 omega_L_mod=omega_L, omega_T_mod=omega_T):
353     sigma = np.asarray(sigma, dtype=np.complex128)
354     lattice = eps_inf_mod * (
355         (omega_L_mod**2 - sigma**2 - 1j * gamma_L * sigma)
356         / (omega_T_mod**2 - sigma**2 - 1j * gamma_T * sigma)
357     )
358     drude = (omega_p**2) / (sigma**2 + 1j * gamma_p * sigma)
359     return lattice - drude
360
361 def get_I_modified(sigma, d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2,
362 eps_inf_mod=eps_inf, omega_L_mod=omega_L, omega_T_mod=omega_T):
363     sigma = np.asarray(sigma, dtype=np.complex128)
364     n1 = np.sqrt(get_epsilon_modified(sigma, gamma_L, gamma_T, omega_p1, gamma_p1, eps_inf_mod,
365 omega_L_mod, omega_T_mod))
366     n2 = np.sqrt(get_epsilon_modified(sigma, gamma_L, gamma_T, omega_p2, gamma_p2, eps_inf_mod,
367 omega_L_mod, omega_T_mod))
368
369     r01s, r01p, t01s, t01p, theta2 = fresnel_rt_and_theta2(n0, n1, theta1)
370     r12s, r12p, t12s, t12p, _ = fresnel_rt_and_theta2(n1, n2, theta2)
371     r10s, r10p, t10s, t10p, _ = fresnel_rt_and_theta2(n1, n0, theta2)
372
373     d_cm = d_um * 1e-4
374     delta = 4 * np.pi * sigma * d_cm * n1 * np.cos(theta2)
375
376     A_s = r01s + t01s * r12s * t10s * np.exp(1j * delta)
377     A_p = r01p + t01p * r12p * t10p * np.exp(1j * delta)
378
379     R = (np.abs(A_s) ** 2 + np.abs(A_p) ** 2) / 2.0
380     return np.real_if_close(R)
381
382 def model_percent_modified(sigma, params, eps_inf_mod=eps_inf, omega_L_mod=omega_L, omega_T_mod=
383 omega_T):
384     d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2 = params
385     R = get_I_modified(sigma, d_um, gamma_L, gamma_T, omega_p1, gamma_p1, omega_p2, gamma_p2,
386 eps_inf_mod, omega_L_mod, omega_T_mod)
387     return 100.0 * R
388
389 def loss_modified(params, eps_inf_mod=eps_inf, omega_L_mod=omega_L, omega_T_mod=omega_T):

```

```

384     loss1 = loss_single_modified(sigma_data1, y_data1, params, eps_inf_mod, omega_L_mod,
omega_T_mod)
385     loss2 = loss_single_modified(sigma_data2, y_data2, params, eps_inf_mod, omega_L_mod,
omega_T_mod)
386     if loss1 >= 1e12 or loss2 >= 1e12:
387         return 1e12
388     return (loss1 + loss2) / 2.0
389
390 def loss_single_modified(sigma_data, y_data, params, eps_inf_mod=eps_inf, omega_L_mod=omega_L,
omega_T_mod=omega_T):
391     y_hat = model_percent_modified(sigma_data, params, eps_inf_mod, omega_L_mod, omega_T_mod)
392     if not np.all(np.isfinite(y_hat)):
393         return 1e12
394     resid = y_hat - y_data
395     return float(np.mean(resid**2))
396
397 # 进行参数扰动分析
398 param_variations = {
399     'eps_inf_minus': eps_inf * (1 - variation),
400     'eps_inf_plus': eps_inf * (1 + variation),
401     'omega_L_minus': omega_L * (1 - variation),
402     'omega_L_plus': omega_L * (1 + variation),
403     'omega_T_minus': omega_T * (1 - variation),
404     'omega_T_plus': omega_T * (1 + variation)
405 }
406
407 thickness_variations = []
408 labels = []
409
410 for name, value in param_variations.items():
411     if 'eps_inf' in name:
412         result = minimize(lambda x: loss_modified(x, eps_inf_mod=value),
413                           x0=best_params, method="L-BFGS-B", bounds=bounds)
414     elif 'omega_L' in name:
415         result = minimize(lambda x: loss_modified(x, omega_L_mod=value),
416                           x0=best_params, method="L-BFGS-B", bounds=bounds)
417     elif 'omega_T' in name:
418         result = minimize(lambda x: loss_modified(x, omega_T_mod=value),
419                           x0=best_params, method="L-BFGS-B", bounds=bounds)
420
421     thickness_change = (result.x[0] - baseline_thickness) / baseline_thickness * 100
422     thickness_variations.append(thickness_change)
423     labels.append(name.replace('_', ' '))
424
425 # 绘制参数灵敏度分析图
426 plt.figure(figsize=(10, 6))
427 colors = ['#e66d50', '#297270', '#e66d50', '#297270', '#e66d50', '#297270']
428 bars = plt.bar(labels, thickness_variations, color=colors, alpha=0.7)
429 plt.axhline(0, color='black', linestyle='--', alpha=0.5)
430 plt.ylabel('薄膜厚度变化 (%)', fontsize=16)
431 plt.title(f'参数灵敏度分析 (扰动幅度: ±{variation*100}%)', fontsize=18)
432 plt.xticks(rotation=45, fontsize=12)
433 plt.tick_params(labelsize=14)
434 for bar, val in zip(bars, thickness_variations):
435     plt.text(bar.get_x() + bar.get_width()/2.0, val + (0.01 if val >= 0 else -0.03),
436             f'{val:.3f}%', ha='center', va='bottom' if val >= 0 else 'top', fontsize=12)
437 plt.tight_layout()
438 plt.savefig("参数灵敏度分析.png", dpi=300, bbox_inches="tight")
439 plt.show()

```

```

440
441 # 2. 数据噪声稳健性分析 (蒙特卡洛)
442 print("\n--- 正在进行噪声稳健性分析 ---")
443
444 num_simulations = 100
445 noise_level = 0.5 # 反射率噪声的标准差为0.5%
446 noisy_thicknesses = []
447
448 for i in range(num_simulations):
449     # 为两个数据集添加噪声
450     noisy_y_data1 = y_data1 + np.random.normal(0, noise_level, len(y_data1))
451     noisy_y_data2 = y_data2 + np.random.normal(0, noise_level, len(y_data2))
452
453     # 定义噪声情况下的loss函数
454     def loss_noisy(params):
455         loss1 = loss_single_noisy(sigma_data1, noisy_y_data1, params)
456         loss2 = loss_single_noisy(sigma_data2, noisy_y_data2, params)
457         if loss1 >= 1e12 or loss2 >= 1e12:
458             return 1e12
459         return (loss1 + loss2) / 2.0
460
461     def loss_single_noisy(sigma_data, y_data_noisy, params):
462         y_hat = model_percent(sigma_data, params)
463         if not np.all(np.isfinite(y_hat)):
464             return 1e12
465         resid = y_hat - y_data_noisy
466         return float(np.mean(resid**2))
467
468     # 进行拟合
469     result = minimize(loss_noisy, x0=best_params, method="L-BFGS-B", bounds=bounds)
470     noisy_thicknesses.append(result.x[0])
471     print(f"\r模拟 {i+1}/{num_simulations}", end="")
472
473 print("\n模拟完成。")
474
475 # 计算统计量
476 thickness_changes = [(t - baseline_thickness) / baseline_thickness * 100 for t in
477     noisy_thicknesses]
478 mean_change = np.mean(thickness_changes)
479 std_change = np.std(thickness_changes)
480
481 # 绘制噪声稳健性分析图
482 plt.figure(figsize=(10, 6))
483 plt.hist(thickness_changes, bins=20, color="#e9f4a3", edgecolor='black', alpha=0.7)
484 plt.axvline(0, color='#e66d50', linestyle='--', label=f'基准厚度: {baseline_thickness:.4f} m')
485 plt.axvline(mean_change, color='#297270', linestyle=':', label=f'噪声下均值变化: {mean_change:.3f}%')
486 plt.xlabel('薄膜厚度变化 (%)', fontsize=16)
487 plt.ylabel('频数', fontsize=16)
488 plt.title(f'噪声稳健性分析 (N={num_simulations}, 噪声={noise_level}%)', fontsize=18)
489 plt.legend(fontsize=14)
490 plt.tick_params(labelsize=14)
491 plt.text(0.02, 0.95, f'标准差: {std_change:.3f}%', transform=plt.gca().transAxes,
492     bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8), fontsize=12)
493 plt.tight_layout()
494 plt.savefig("噪声稳健性分析.png", dpi=300, bbox_inches="tight")
495 plt.show()
496
497 print("=" * 50)

```

```

497     print("灵敏度分析完成")
498     print(f"基准薄膜厚度: {baseline_thickness:.4f} m")
499     print(f"噪声影响 - 均值变化: {mean_change:.3f}%, 标准差: {std_change:.3f}%")
500     print("=" * 50)
501
502     print("=" * 50)
503     print("图片已保存: ")
504     print("- 数据集1_拟合结果.png")
505     print("- 数据集2_拟合结果.png")
506     print("- 数据集1_残差分析.png")
507     print("- 数据集2_残差分析.png")
508     print("- 折射率_波数依赖性.png")
509     print("- 参数灵敏度分析.png")
510     print("- 噪声稳健性分析.png")
511     print("=" * 50)

```

Listing 1 问题二求解核心代码

B.2 问题三干涉极值求解的 Python 关键代码 (q3-interfere.py)

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import zhplot
4
5
6  class SiThicknessEstimator:
7      def __init__(self):
8          print("初始化基于干涉极值点法的厚度估算器...")
9
10     def calculate_n_si(self, nu_cm):
11         nu = np.asarray(nu_cm, dtype=float)
12         B1, C1 = 10.6684293, 0.0909121907
13         B2, C2 = 0.0030434748, 1.287660172
14         B3, C3 = 1.54133408, 1.218816e6
15
16         n2 = (
17             1.0
18             + (B1 * 1e8) / (1e8 - C1 * nu**2)
19             + (B2 * 1e8) / (1e8 - C2 * nu**2)
20             + (B3 * 1e8) / (1e8 - C3 * nu**2)
21         )
22         return np.sqrt(n2)
23
24     def estimate_thickness(self, extrema_points, angle_deg, m_range=range(1, 40)):
25         print(f"\n{' '*25} 开始估算厚度 (入射角: {angle_deg}°) {' '*25}")
26
27         best_m_start = -1
28         min_std_dev = float('inf')
29         best_results = {}
30
31         all_std_devs = []
32
33         for m_start in m_range:
34             d_values = []
35             for i, point in enumerate(extrema_points):
36                 wavenumber = point['wavenumber']
37                 if extrema_points[0]['type'] == 'peak':
38                     order = m_start + i / 2.0

```

```

39         else:
40             order = m_start + (i+1) / 2.0
41
42
43         n1 = self.calculate_n_si(wavenumber)
44         theta0_rad = np.radians(angle_deg)
45
46         theta1_rad = np.arcsin(np.sin(theta0_rad) / n1)
47
48         d = order / (2 * n1 * np.cos(theta1_rad) * wavenumber * 1e-4)
49         d_values.append(d)
50
51     std_dev = np.std(d_values)
52     all_std_devs.append(std_dev)
53
54     if std_dev < min_std_dev:
55         min_std_dev = std_dev
56         best_m_start = m_start
57         best_results = {
58             'best_m_start': best_m_start,
59             'avg_thickness_um': np.mean(d_values),
60             'min_std_dev': min_std_dev,
61             'thickness_values': d_values,
62         }
63
64     self.plot_std_dev_vs_m(m_range, all_std_devs, best_results, angle_deg)
65
66     return best_results
67
68 def plot_std_dev_vs_m(self, m_range, std_devs, best_results, angle_deg):
69     fig, ax = plt.subplots(figsize=(8, 5))
70     ax.plot(m_range, std_devs, 'o-', markersize=4, label='厚度计算值的标准差')
71     ax.axvline(best_results['best_m_start'], color='r', linestyle='--',
72               label=f'最优解 m = {best_results["best_m_start"]}')
73     ax.set_xlabel('假定的初始干涉级数 (m)', fontsize=16)
74     ax.set_ylabel('计算厚度 d 的标准差 (m)', fontsize=16)
75     ax.set_title(f'厚度一致性分析 (入射角 {angle_deg}°)', fontsize=18, fontweight='bold')
76     ax.grid(True)
77     ax.legend(fontsize=14)
78     ax.tick_params(axis='both', which='major', labelsize=14)
79     plt.tight_layout()
80     plt.savefig(f'厚度一致性分析_{angle_deg}度.png', dpi=300)
81     print(f" 厚度一致性分析图已保存。")
82     plt.show()
83
84 if __name__ == '__main__':
85     extrema_points_3 = [
86         {'wavenumber': 750.1736, 'reflectance': 70.957828}, {'wavenumber': 940.6096, 'reflectance':
87         0.134385},
88         {'wavenumber': 1105.494, 'reflectance': 55.786779}, {'wavenumber': 1312.804, 'reflectance':
89         11.654351},
90         {'wavenumber': 1508.061, 'reflectance': 39.33095}, {'wavenumber': 1722.603, 'reflectance':
91         20.771007},
92         {'wavenumber': 1927.02, 'reflectance': 33.108994}, {'wavenumber': 2144.937, 'reflectance':
93         24.514375},
94         {'wavenumber': 2378.764, 'reflectance': 30.360874}, {'wavenumber': 2564.861, 'reflectance':
95         26.210183},
96         {'wavenumber': 2779.885, 'reflectance': 29.43295}, {'wavenumber': 2993.462, 'reflectance':
97         27.052822},

```

```

92     {'wavenumber': 3203.665, 'reflectance': 28.874511}, {'wavenumber': 3465.455, 'reflectance':
27.624775},
93     {'wavenumber': 3650.587, 'reflectance': 28.630007}, {'wavenumber': 3913.341, 'reflectance':
27.896136},
94 ]
95 for i, point in enumerate(extrema_points_3): point['type'] = 'peak' if i % 2 == 0 else 'trough'
96
97 extrema_points_4 = [
98     {'wavenumber': 574.201, 'reflectance': 3.8458416}, {'wavenumber': 749.2094, 'reflectance':
78.781032},
99     {'wavenumber': 946.395, 'reflectance': 0.3493311}, {'wavenumber': 1108.386, 'reflectance':
60.657749},
100     {'wavenumber': 1324.375, 'reflectance': 13.301342}, {'wavenumber': 1519.15, 'reflectance':
43.048557},
101     {'wavenumber': 1736.102, 'reflectance': 22.879293}, {'wavenumber': 1941.966, 'reflectance':
36.174206},
102     {'wavenumber': 2160.847, 'reflectance': 26.795459}, {'wavenumber': 2387.442, 'reflectance':
33.288474},
103     {'wavenumber': 2587.038, 'reflectance': 28.637388}, {'wavenumber': 2802.544, 'reflectance':
32.164834},
104     {'wavenumber': 3035.889, 'reflectance': 29.60452}, {'wavenumber': 3245.127, 'reflectance':
31.675433},
105     {'wavenumber': 3450.027, 'reflectance': 30.238529}, {'wavenumber': 3676.14, 'reflectance':
31.571544}
106 ]
107 for i, point in enumerate(extrema_points_4): point['type'] = 'trough' if i % 2 == 0 else 'peak'
108
109 estimator = SiThicknessEstimator()
110
111 results_3 = estimator.estimate_thickness(extrema_points_3, angle_deg=10, m_range=range(1, 15))
112
113 results_4 = estimator.estimate_thickness(extrema_points_4, angle_deg=15, m_range=range(1, 15))
114
115 d3 = results_3['avg_thickness_um']
116 d4 = results_4['avg_thickness_um']
117
118 avg_d = (d3 + d4) / 2
119 diff_percent = abs(d3 - d4) / avg_d * 100 if avg_d > 0 else 0

```

Listing 2 问题三求解核心代码

B.3 问题三基于色散矫正傅里叶变换求解的 Python 关键代码 (q3-fft.py)

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import zhplot
5 COLOR_PALETTE = {
6     "dark_teal": "#274753", # 深青色
7     "medium_teal": "#297270", # 中青色
8     "teal": "#299d8f", # 青色
9     "sage_green": "#8ab07c", # 鼠尾草绿
10    "gold": "#e7c66b", # 金色
11    "orange": "#f3a361", # 橙色
12    "coral": "#e66d50", # 珊瑚色
13 }
14 def sellmeier_silicon(wavenumber_cm):
15     wavelength_um = 1 / (wavenumber_cm * 1e-4)

```

```

16
17 B1, B2, B3 = 10.6684293, 0.0030434748, 1.54133408
18 C1, C2, C3 = 0.09091219, 1.28766018, 1218816
19
20 lambda_sq = wavelength_um**2
21
22 n_sq = (
23     1
24     + (B1 * lambda_sq) / (lambda_sq - C1)
25     + (B2 * lambda_sq) / (lambda_sq - C2)
26     + (B3 * lambda_sq) / (lambda_sq - C3)
27 )
28
29 return np.sqrt(n_sq)
30
31
32 def quadratic_peak_interpolation(y_values, x_values):
33     if len(y_values) != 3 or len(x_values) != 3:
34         raise ValueError("需要且仅需要三个点来进行二次插值。")
35
36     A = np.vstack([x_values**2, x_values, np.ones(3)]).T
37     a, b, c = np.linalg.solve(A, y_values)
38
39     return -b / (2 * a), (a, b, c)
40
41
42 def perform_fft_analysis(filename, angle_deg, num_points=8192):
43
44     df = pd.read_excel(filename, header=None)
45     wavenumber_raw = pd.to_numeric(df.iloc[:, 0], errors="coerce").values
46     reflectance_raw = pd.to_numeric(df.iloc[:, 1], errors="coerce").values
47     valid_mask = ~np.isnan(wavenumber_raw) & ~np.isnan(reflectance_raw)
48     wavenumber_raw, reflectance_raw = (
49         wavenumber_raw[valid_mask],
50         reflectance_raw[valid_mask],
51     )
52
53     n_values = sellmeier_silicon(wavenumber_raw)
54     sin_theta_i_sq = np.sin(np.deg2rad(angle_deg)) ** 2
55     x_domain_nonuniform = 2 * wavenumber_raw * np.sqrt(n_values**2 - sin_theta_i_sq)
56
57     x_domain_uniform = np.linspace(
58         x_domain_nonuniform.min(), x_domain_nonuniform.max(), num_points
59     )
60     reflectance_uniform_in_x = np.interp(
61         x_domain_uniform, x_domain_nonuniform, reflectance_raw
62     )
63
64     window = np.hanning(num_points)
65     windowed_signal = (
66         reflectance_uniform_in_x - np.mean(reflectance_uniform_in_x)
67     ) * window
68
69     fft_result = np.fft.fft(windowed_signal)
70     fft_amplitude = np.abs(fft_result)
71     delta_x = x_domain_uniform[1] - x_domain_uniform[0]
72     fft_freq_d_cm = np.fft.fftfreq(num_points, d=delta_x)
73
74     min_d_um = 2.0

```

```

75 positive_mask = (fft_freq_d_cm * 1e4) > min_d_um
76 peak_idx_rough = np.argmax(fft_amplitude[positive_mask])
77 peak_idx_global = np.where(positive_mask)[0][peak_idx_rough]
78 indices_to_fit = np.array(
79     [peak_idx_global - 1, peak_idx_global, peak_idx_global + 1]
80 )
81 d_values_to_fit = fft_freq_d_cm[indices_to_fit]
82 amp_values_to_fit = fft_amplitude[indices_to_fit]
83 d_cm_precise, parabola_coeffs = quadratic_peak_interpolation(
84     amp_values_to_fit, d_values_to_fit
85 )
86 d_um = d_cm_precise * 1e4
87
88
89 return {
90     "filename": filename,
91     "d_um": d_um,
92     "x_domain_uniform": x_domain_uniform,
93     "reflectance_uniform_in_x": reflectance_uniform_in_x,
94     "fft_freq_d_um": fft_freq_d_cm * 1e4,
95     "fft_amplitude": fft_amplitude,
96     "positive_mask": positive_mask,
97     "d_values_to_fit_um": d_values_to_fit * 1e4,
98     "amp_values_to_fit": amp_values_to_fit,
99     "parabola_coeffs": parabola_coeffs,
100 }
101
102
103 def plot_fft_details(ax, data, title):
104     ax.plot(
105         data["fft_freq_d_um"][data["positive_mask"]],
106         data["fft_amplitude"][data["positive_mask"]],
107         "o--",
108         color="#4965b0",
109         markersize=6,
110         linewidth=2,
111         label="FFT 频谱（离散点）",
112     )
113
114     ax.fill_between(
115         data["fft_freq_d_um"][data["positive_mask"]],
116         data["fft_amplitude"][data["positive_mask"]],
117         alpha=0.3,
118         color="#cfeaf1",
119         label="FFT 频谱区域"
120     )
121
122     ax.plot(
123         data["d_values_to_fit_um"],
124         data["amp_values_to_fit"],
125         "o",
126         color="#a30543",
127         markersize=10,
128         label="用于二次插值的三个关键点",
129     )
130
131     a, b, c = data["parabola_coeffs"]
132     a_um = a / 1e8
133     b_um = b / 1e4

```

```

134 d_fine_grid_um = np.linspace(
135     data["d_values_to_fit_um"].min(), data["d_values_to_fit_um"].max(), 100
136 )
137 parabola_y = a_um * d_fine_grid_um**2 + b_um * d_fine_grid_um + c
138 ax.plot(
139     d_fine_grid_um,
140     parabola_y,
141     "-",
142     color="#f36f43",
143     linewidth=3,
144     label=f"二次插值拟合抛物线",
145 )
146
147 ax.axvline(data["d_um"], color="crimson", ls="--", lw=3)
148
149 ax.set_title(title, fontsize=16)
150 ax.set_xlabel("厚度 d ( m)", fontsize=12)
151 ax.set_ylabel("FFT 振幅", fontsize=12)
152
153 zoom_range = 1.5
154 ax.set_xlim(data["d_um"] - zoom_range, data["d_um"] + zoom_range)
155
156 ax.grid(True, linestyle="--", alpha=0.6)
157 ax.legend(fontsize=10)
158
159
160 if __name__ == "__main__":
161     data_f3 = perform_fft_analysis(filename="./attach/3.xlsx", angle_deg=10)
162     data_f4 = perform_fft_analysis(filename="./attach/4.xlsx", angle_deg=15)
163
164     plt.rcParams["font.sans-serif"] = ["SimHei"] # 设置中文字体
165     plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题
166
167     fig1, ax1 = plt.subplots(1, 1, figsize=(10, 6))
168
169     ax1.plot(
170         data_f3["x_domain_uniform"],
171         data_f3["reflectance_uniform_in_x"],
172         label="在新坐标域(x)重采样后的信号",
173         color="teal",
174         linewidth=2.5
175     )
176     ax1.set_title("经坐标变换后的完美周期信号 (以附件3为例)", fontsize=20, fontweight='bold')
177     ax1.set_xlabel("新坐标  $x = 2 \sqrt{n^2 - \sin^2}$  (cm-1)", fontsize=16)
178     ax1.set_ylabel("反射率 (%)", fontsize=16)
179     ax1.grid(True, linestyle="--", alpha=0.6)
180     ax1.legend(fontsize=14)
181     ax1.tick_params(axis='both', which='major', labelsize=14)
182
183     plt.tight_layout()
184     plt.savefig("坐标变换后的周期信号.png", dpi=300, bbox_inches='tight')
185     plt.close()
186
187     fig2, axes = plt.subplots(2, 1, figsize=(10, 10))
188
189     plot_fft_details(axes[0], data_f3, title="附件3 (10° 入射角) FFT频谱分析")
190
191     plot_fft_details(axes[1], data_f4, title="附件4 (15° 入射角) FFT频谱分析")
192

```

```
193     fig2.tight_layout(pad=3.0)
194     plt.savefig("FFT频谱分析结果.png", dpi=300, bbox_inches='tight')
195     plt.close()
```

Listing 3 问题三求解核心代码